Politechnika Warszawska

WYDZIAŁ ELEKTRYCZNY

Instytut Elektrotechniki Teoretycznej i Systemów Informacyjno-Pomiarowych

Praca dyplomowa inżynierska

na kierunku Informatyka Stosowana

w specjalności Inżynieria Oprogramowania

Platforma internetowa typu SaaS do zarządzania kursami

Mikołaj Pątkowski

numer albumu 325502

promotor

mgr inż. Marek Wdowiak

WARSZAWA 2026

Platforma internetowa typu SaaS do zarządzania kursami

Streszczenie

Praca ma na celu przedstawienie dokumentacji systemu Coursy, służącego do tworzenia oraz zarządzania platformami z kursami video. Aplikacja została zaprojektowana w oparciu o architekturę mikroserwisową, hybrydowe renderowanie widoków oraz model biznesowy SaaS.

Zaprojektowano architekturę mikrousługową składającą się z pięciu niezależnych serwisów odpowiedzialnych za uwierzytelnianie, zarządzanie użytkownikami, obsługę platform, kursów oraz materiałów edukacyjnych. System został wdrożony w środowisku kontenerowym z wykorzystaniem orkiestracji klastrowej.

Zrealizowane rozwiązanie umożliwia jednoczesną obsługę wielu izolowanych platform edukacyjnych, z których każda może być niezależnie personalizowana wizualnie i zarządzana przez odrębnego najemcę. Wielonajemcowość zaimplementowano poprzez separację danych na poziomie bazy relacyjnej z wykorzystaniem identyfikatorów platform, co zapewnia izolację zasobów między klientami przy jednoczesnym współdzieleniu infrastruktury. System wspiera różnorodne typy materiałów edukacyjnych, w tym strumieniowanie treści video z adaptacyjnym doborem jakości oraz interaktywne komponenty takie jak quizy i sformatowane treści tekstowe. Mechanizm uwierzytelniania oparty o żetony JWT zapewnia bezpieczny dostęp do zasobów z uwzględnieniem kontekstu platformy i roli użytkownika. Routing żądań realizowany jest poprzez subdomeny, gdzie każda platforma najemcy funkcjonuje pod unikalnym adresem, co dodatkowo wzmacnia wrażenie odrębności poszczególnych instancji systemu.

Praca dowodzi, że architektura mikrousługowa w połączeniu z wzorcem wielonajemnościowości pozwala na efektywną separację danych między klientami przy jednoczesnym zachowaniu elastyczności rozwoju systemu. Zastosowanie izolacji na poziomie bazy danych oraz mechanizmów uwierzytelniania kontekstowego zapewnia bezpieczeństwo i skalowalność rozwiązania typu SaaS.

Słowa kluczowe: SaaS, architektura mikroserwisowa, wielonajemcowość, renderowanie hybrydowe, streaming video, Spring Boot, Next.js, Kubernetes

SaaS online platform for course management

Abstract

The aim of this thesis is to present the documentation of the Coursy system, which is used to create and manage platforms with video courses. The application was designed based on microservice architecture, hybrid view rendering, and the SaaS business model.

A microservice architecture was designed, consisting of five independent services responsible for authentication, user management, platform support, courses, and educational materials. The system was implemented in a container environment using cluster orchestration.

The implemented solution enables the simultaneous operation of multiple isolated educational platforms, each of which can be independently customized visually and managed by a separate tenant. Multi-tenant support was implemented by separating data at the relational database level using platform identifiers, which ensures resource isolation between clients while sharing infrastructure. The system supports a variety of educational materials, including video streaming with adaptive quality selection and interactive components such as quizzes and formatted text content. The JWT token-based authentication mechanism ensures secure access to resources, taking into account the platform context and user role. Request routing is performed through subdomains, where each tenant platform operates under a unique address, which further reinforces the impression of separate system instances.

The work proves that microservice architecture combined with the multi-tenancy pattern allows for effective data separation between clients while maintaining flexibility in system development. The use of database-level isolation and JWT mechanisms ensures the security and scalability of the SaaS solution.

Keywords: SaaS, microservice architecture, multi-tenancy, hybrid rendering, video streaming, Spring Boot, Next.js, Kubernetes

Spis treści

Wstęp **9**

1 Architektura systemów SaaS wielonajemcowych **11**
1.1 Modele wielonajemcowości w aplikacjach SaaS 12
1.2 Architektura mikroserwisowa i jej zastosowanie w SaaS 13
1.3 Strategie renderowania w aplikacjach webowych 15

2 Projektowanie platformy do dystrybucji kursów video **17**
2.1 Wymagania funkcjonalne i niefunkcjonalne 18
2.2 Przypadki użycia 20
2.3 Model dziedziny 25
2.4 Wybór technologii 26

3 Implementacja systemu Coursy **29**
3.1 Podział na mikroserwisy 29
3.2 Model danych i strategia bazy danych 30
3.3 System autentykacji i autoryzacji 36
3.4 Identyfikacja najemcy i routing żądań 38
3.5 Streaming video z użyciem protokołu HLS 38
3.6 Obsługa warstwy frontend 42
3.7 Orkiestracja 43

4 Weryfikacja i demonstracja działania systemu **45**
4.1 Środowisko testowe 45
4.2 Demonstracja kluczowych funkcjonalności 45

5 Podsumowanie i wnioski **57**

Bibliografia **61**

Wstęp

Współczesne aplikacje webowe coraz częściej projektowane są w oparciu o architekturę mikroserwisową, która umożliwia podział złożonego systemu na mniejsze, niezależnie wdrażane komponenty. Podejście to oferuje szereg korzyści, takich jak większa skalowalność, łatwiejsze utrzymanie oraz możliwość wykorzystania różnych technologii w poszczególnych serwisach. Jednocześnie model dystrybucji oprogramowania Software as a Service (SaaS) w wariancie wielonajemcowym stanowi wyzwanie architektoniczne, wymagające zapewnienia separacji danych oraz bezpieczeństwa przy współdzieleniu wspólnej infrastruktury przez wielu klientów.

Celem niniejszej pracy jest zaprojektowanie oraz implementacja systemu SaaS opartego o architekturę mikroserwisową z odpowiednią separacją odpowiedzialności biznesowej. Dodatkowym założeniem jest również wykorzystanie oraz zbadanie możliwości hybrydowego generowania widoków po stronie serwera oraz klienta, które jest oferowane przez nowoczesne frameworki frontendowe. Jako domenę biznesową wybrano kreator platform z kursami video – system umożliwiający najemcom (organizacjom, firmom szkoleniowym) tworzenie i zarządzanie własnymi brandowanymi platformami edukacyjnymi, na których użytkownicy końcowi mogą korzystać z kursów video. Taka domena pozwala na zademonstrowanie kluczowych wyzwań architektonicznych wielonajemcowego systemu SaaS: separacji danych między najemcami, routingu opartego o subdomeny, personalizacji interfejsu per platforma oraz zarządzania zasobami multimedialnymi w środowisku mikroserwisowym. Zaimplementowany system nosi nazwę Coursy.

Zakres pracy obejmuje zaprojektowanie architektury systemu składającego się z niezależnych mikroserwisów odpowiedzialnych za uwierzytelnianie, zarządzanie użytkownikami, zarządzanie platformami najemców, zarządzanie kursami oraz obsługę materiałów wewnątrz kursów. System wykorzystuje Spring Boot z językiem Kotlin po stronie backendu, Next.js z TypeScript po stronie frontendu oraz Kubernetes do orkiestracji kontenerów. Zakres pracy obejmuje również implementację mechanizmów separacji danych w architekturze wielonajemcowej, routingu opartego o subdomeny oraz systemu uwierzytelniania i autoryzacji z wykorzystaniem JWT. Poza zakresem pracy pozostają funkcjonalności takie jak płatności, streaming na żywo lub zaawansowana analityka.

Praca składa się z 5 rozdziałów. Pierwszy rozdział *Architektura systemów SaaS wielonajemcowych* zawiera definicję oraz opis modelu SaaS stosowanego w niniejszej pracy oraz rozwiązania architektoniczne pozwalające na skuteczną implementację wielonajemcowości. Dodatkowo zostały

zaprezentowane sposoby generowania widoków w aplikacjach webowych, ze szczególnym uwzględnieniem renderowania hybrydowego oferowanego przez współczesne frameworki JavaScript.

Rozdział drugi *Projektowanie platformy do dystrybucji kursów video* opisuje proces analizy wymagań funkcjonalnych i niefunkcjonalnych systemu. Następnie prezentuje model dziedziny, na którym został oparty system Coursy. Zawiera również pełną listę wykorzystanych technologii wraz z uzasadnieniem ich wyboru.

W trzecim rozdziale *Implementacja systemu Coursy* zaprezentowano szczegóły techniczne realizowanej aplikacji. Rozdział ten zawiera informacje o podziale mikroserwisowym systemu, strukturze bazodanowej, obsłudze frontendu oraz rozwiązaniach problemów takich jak routing żądań, uwierzytelnianie, przetwarzanie plików video oraz orkiestracja w środowisku Kubernetes.

Rozdział czwarty *Weryfikacja i demonstracja działania systemu* prezentuje utworzony system poprzez demonstrację kluczowych przypadków użycia z perspektywy zarówno najemcy, jak i użytkownika końcowego. Weryfikacja obejmuje potwierdzenie poprawności działania mechanizmów wielonajemcowości, separacji danych, routingu opartego o subdomeny oraz funkcjonalności zarządzania kursami i materiałami. Rozdział zawiera również zrzuty ekranu ilustrujące interfejs użytkownika.

Ostatni rozdział *Podsumowanie i wnioski* przedstawia ocenę realizacji wyznaczonych celów pracy oraz refleksje dotyczące napotkanych trudności implementacyjnych. Omawia wnioski wynikające z procesu projektowania i implementacji systemu mikroserwisowego, a także wskazuje potencjalne kierunki dalszego rozwoju systemu Coursy.

Praca dokumentuje cały proces – od analizy wymagań przez projektowanie architektury, implementację poszczególnych komponentów, aż po weryfikację działania systemu i refleksję nad napotykanymi problemami. Przedstawione rozwiązania architektoniczne oraz doświadczenia implementacyjne mogą stanowić punkt odniesienia dla podobnych projektów wykorzystujących architekturę mikroserwisową w modelu SaaS wielonajemcowego. Zrealizowany system dowodzi praktycznej wykonalności połączenia tych podejść architektonicznych w kontekście aplikacji dystrybucji treści edukacyjnych.

Rozdział 1

Architektura systemów SaaS wielonajemcowych

SaaS (Software as a Service) to model biznesowy, w którym użytkownik uzyskuje dostęp do oprogramowania przez subskrypcję/abonament. Główną różnicą pomiędzy konwencjonalnym modelem dystrybucji jest to, że oprogramowanie nie jest instalowane lokalnie na maszynie klienta, ale dostępne jako aplikacja webowa w przeglądarce. Model subskrypcyjny opiera się na PAYG (Pay as you go). Oznacza to, że klient cyklicznie płaci za usługę wtedy, kiedy jej używa, a nie poprzez jednorazowy zakup.

W ostatnich latach można zauważyć znaczący wzrost zainteresowania tym modelem biznesowym [14]. Jest to spowodowane kilkoma czynnikami:

- Rozwój technologiczny - największe firmy technologiczne przyczyniły się do ulepszenia technologii webowych oraz internetu, który jest podstawą architektoniczną dla produktów SaaS.
- Łatwiejsze aktualizacje i dystrybucja - produkt udostępniany internetowo jest prostszy w zarządzaniu w stosunku do fizycznych kopii, co redukuje koszty rozwoju oraz zapisywania na fizycznych nośnikach (np. CD).
- Scentralizowane bezpieczeństwo - wszystko jest zarządzane poprzez wydajne oraz bezpieczne usługi dostawców chmurowych, a użytkownicy nie muszą się martwić o zapewnienie bezpieczeństwa. Oprogramowanie działa po stronie serwera, nie na ich maszynach.
- Niższe koszty początkowe dla użytkowników - do korzystania z SaaS nie trzeba inwestować w odpowiedni sprzęt, a opłata subskrypcyjna jest również niższa niż jednorazowy zakup oprogramowania.
- Minimalizowanie piractwa - aplikacja typu SaaS jest uruchamiana na serwerze i niemożliwe jest jej skopiowanie bez wykradnięcia kodu źródłowego bezpośrednio z infrastruktury serwera.

Innymi charakterystycznymi modelami usług sieciowych są PaaS oraz IaaS [15]. PaaS (Platform as a Service) to usługa polegająca na dostarczeniu platformy obliczeniowej, pozwalającej między innymi na rozwój oraz wdrażanie aplikacji. Klient uzyskuje dostęp do środowiska, nie ma natomiast nad nim pełnej kontroli. Przykładowo, nie jest w stanie zmienić zainstalowanego systemu operacyjnego. IaaS

(Infrastructure as a Service) z kolei pozwala na wynajęcie infrastruktury wirtualnej lub fizycznej do zagospodarowania przez użytkownika. Klient w tym przypadku ma pełną kontrolę nad udostępnionymi maszynami.

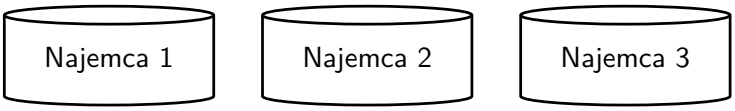
1.1 Modele wielonajemcowości w aplikacjach SaaS

Charakterystyczną cechą SaaS jest wielonajemcowość (Multitenancy). Oznacza ona to, że pojedyncza instancja aplikacji służy jednocześnie wielu klientom (najemcom), którzy mają wrażenie, jakby korzystali z różnych aplikacji [8]. To rozwiązanie implementuje się najczęściej na poziomie bazy danych, która wspiera rozróżnienie odpowiedniej wersji usługi w zależności od najemcy. Warto zauważyć, że aplikacje typu SaaS często służą do obsługi biznesu najemców, którzy z kolei mają swoich własnych użytkowników końcowych. W takim modelu występuje hierarchia zaprezentowana na rysunku 1. Z perspektywy architektury systemu, termin *klient* odnosi się do najemcy, nie do jego użytkowników końcowych.

Rysunek 1. Hierarchia użytkowników SaaS

Wspólna aplikacja, osobna baza danych

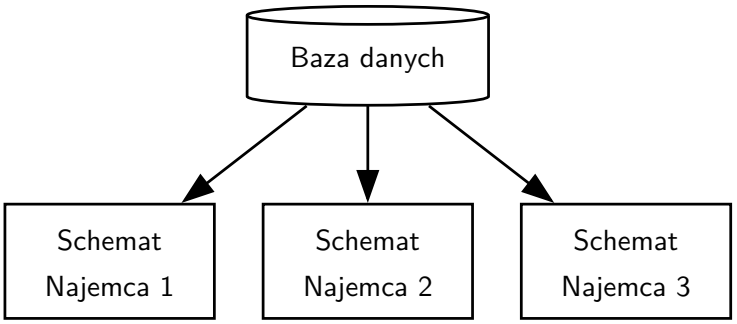
W tym modelu każdy najemca posiada osobną bazę danych. Jest to najbezpieczniejsze rozwiązanie wspierające pełną izolację aplikacji. W przypadku włamania do bazy danych, dane maksymalnie jednego najemcy mogą zostać ukradzione. Jednocześnie jest to również najdroższe rozwiązanie.



Rysunek 2. Wspólna aplikacja, osobna baza danych

Wspólna aplikacja i baza danych, osobne schematy

W tym modelu rozróżnienie odbywa się na poziomie schematów bazy danych (Database schema). Każdy najemca ma swój własny schemat zawierający jego tabele. Z logicznego punktu widzenia aplikacje najemców są odizolowane od siebie tak samo, jak w przypadku osobnych baz danych. Jednakże w przypadku ataku na jedną bazę danych istnieje ryzyko skradzenia danych wszystkich najemców.



Rysunek 3. Wspólna aplikacja i baza danych, osobne schematy

Wspólna aplikacja i tabela

Najprostsze rozwiązanie, w którym dane wszystkich najemców przechowywane są we współdzielonych tabelach. Informacja o tym, której aplikacji dotyczy dany wiersz, jest przechowywana jako jedna z kolumn. To podejście jest najmniej optymalne pod względem bezpieczeństwa oraz wydajności (tabele zawierające dane wszystkich klientów jednocześnie stają się bardzo obszerne). Dodatkowo pojedyncza tabela w bazie relacyjnej ogranicza elastyczność przechowywanych danych, co nie jest jednak istotne w przypadku, gdy wszystkie aplikacje przewidują ten sam schemat danych. Jednakże jest to jednocześnie najtańsze rozwiązanie w utrzymaniu oraz najmniej skomplikowane w implementacji.

id	**id_najemcy**	dane
1	**621**	ABC
2	**978**	XYZ
3	**835**	HJK

Rysunek 4. Wspólna tabela

1.2 Architektura mikroserwisowa i jej zastosowanie w SaaS

Architektura mikroserwisowa to podejście w projektowaniu oprogramowania, według którego jedna główna aplikacja składa się z wielu pojedynczych, małych aplikacji (mikroserwisów) [7]. Te pojedyncze części systemu komunikują się ze sobą poprzez proste protokoły komunikacyjne (HTTP/REST, kolejki komunikatów) i współpracują, realizując kompletny system. Każdy mikroserwis jest odpowiedzialny za pewną konkretną część biznesu, co ma zagwarantować ich izolację.

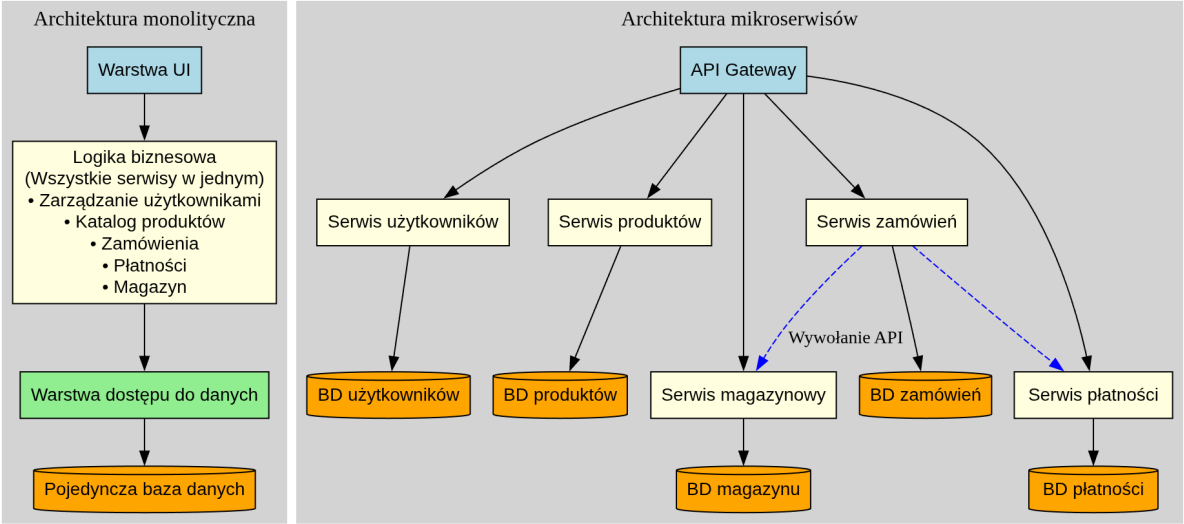
Ten model architektury został zaadoptowany w przemyśle dzięki wielu kluczowym cechom, między innymi:

- Skalowalność - rozdzielenie aplikacji na wiele części składowych pozwala na niezależne skalowanie tego fragmentu systemu, który w danej chwili jest najbardziej obciążony.

- Wykorzystanie zasobów - dostarczana architektura serwerowa jest skuteczniej wykorzystywana przez mikroserwisy, które zużywają tyle zasobów chmurowych, ile aktualnie potrzebują.
- Utrzymanie i rozwój - łatwiej jest wprowadzać zmiany w oprogramowaniu, którego fragmenty są ze sobą luźno połączone. Przy dobrze zaprojektowanych granicach między serwisami, izolacja minimalizuje ryzyko, że modyfikacja jednego komponentu wpłynie negatywnie na pozostałe części systemu. Pozwala to również na współpracę większej liczby programistów, dzieląc ich na mniejsze zespoły oraz przypisując do pracy nad konkretnymi mikroserwisami.
- Łatwiejsze wdrożenia - każdy serwis może wdrażać się niezależnie od pozostałych, ponieważ nie są ze sobą logicznie połączone.
- Odporność na usterki - błąd w jednym mikroserwisie nie jest błędem całego systemu. Dzięki zastosowaniu technik takich jak Circuit-Breaking (pol. *wyłącznik awaryjny*) aplikacja jest w stanie kontynuować pracę pomimo błędów krytycznych poszczególnych jej części, zwłaszcza przy całkowitym wyłączeniu mikroserwisu.

Największą trudnością i wyzwaniem w implementacji tej architektury jest odpowiednie wydzielenie mikroserwisów [13]. Należy na poziomie projektowania oprogramowania zlokalizować obszary biznesowe, które charakteryzują się odpowiednią separacją i niezależnością od pozostałych sfer. Nieprawidłowa identyfikacja tych obszarów i zbyt ziarniste rozdrobnienie aplikacji może skutkować wysoce zależnymi od siebie serwisami, które będą podwójnie implementować wymagania biznesowe. Oprócz tego rośnie również narzut na komunikację międzyserwisową, co ma bezpośrednie przełożenie na wydajność aplikacji. Z kolei wydzielenie serwisów o zbyt obszernej odpowiedzialności minimalizuje zyski pochodzące z tego podejścia architektonicznego. Idealną sytuacją jest wydzielenie mikroserwisów w taki sposób, aby nie musiały się ze sobą w ogóle komunikować. W praktyce jednak jest to nieosiągalne.

Ważnym elementem aplikacji mikroserwisowej jest również API Gateway (pol. *brama API*) [3]. Jest to usługa, która stoi przed całą aplikacją. Służy ona jako fasada, aby wszystkie mikroserwisy były obserwowalne z zewnątrz jako jedno, spójne oprogramowanie. Do API Gateway przychodzą zewnętrzne żądania HTTP, a następnie zostają rozdzielone do właściwych serwisów. Oprócz tego API Gateway może też wstępnie przetwarzać żądania oraz być odpowiedzialne za autentykację i autoryzację.



Rysunek 5. Porównanie architektury monolitycznej i mikroserwisowej

1.3 Strategie renderowania w aplikacjach webowych

Aplikacje SaaS wspierają różne strategie renderowania w przeglądarce, zapewniające odpowiedni zestaw zalet i wad [1]. Frameworki takie jak Next.js pozwalają na budowanie aplikacji korzystających z wielu strategii renderowania jednocześnie, co pozwala na optymalizację aplikacji i wybór najlepszego rozwiązania dla danej strony lub fragmentu systemu. Poniżej zostały zaprezentowane najpopularniejsze strategie używane aktualnie w warstwie frontendowej.

CSR

Client Side Rendering (pol. *renderowanie po stronie klienta*) polega na przesłaniu do przeglądarki minimalnego pliku HTML oraz odpowiedniego skryptu *JavaScript Bundle*, który całkowicie przejmuje kontrolę nad aplikacją. Skrypt jest między innymi odpowiedzialny za zbudowanie DOM, wykonywanie żądań oraz dynamiczność aplikacji webowej. Tego typu podejście jest szeroko stosowane w SPA (Single Page Application), które zakłada obsługę aplikacji bez odświeżeń strony. Wadą tego rozwiązania jest natomiast optymalizacja pod kątem wyszukiwarek internetowych. Kiedy bot indeksujący internet wchodzi na stronę renderowaną całkowicie po stronie klienta, pobiera wybrakowany plik HTML oraz skrypt, który tworzy stronę. Bot jednak nie jest w stanie uruchomić tego skryptu, czego skutkiem jest niemożliwość indeksowania treści strony i pogorszone wyświetlanie w silniku wyszukiwarek.

SSR

Server Side Rendering (pol. *renderowanie po stronie serwera*) to rozwiązanie, w którym cały widok strony tworzony jest przez serwer, a do przeglądarki trafia gotowy plik HTML. Wszystkie potrzebne dane prezentowane na widoku są pobierane przez serwer w trakcie generacji. W tym rozwiązaniu maszyna kliencka ogranicza się tylko do wykonania żądania początkowego, a całe obciążenie ponosi serwer. Strony ładują się również szybciej, ponieważ nie jest wymagane pobieranie obszernego JS Bundle. To rozwiązanie jest dużo lepsze niż CSR pod względem indeksowania przez przeglądarki.

Ograniczeniem natomiast jest interaktywność. Ponieważ strona nie posiada kodu klienckiego, nie może również zawierać interaktywności, w tym np. zachowania UI po naciśnięciu przycisku. Strony generowane przez serwer nie mogą być dynamiczne.

SSG

Static Site Generation (pol. *statyczne tworzenie strony*) generuje gotowe strony HTML już na etapie budowy aplikacji. Jest to rozwiązanie podobne do SSR z tą różnicą, że SSR tworzy stronę za każdym razem, natomiast SSG tylko jeden raz. Ten fakt oznacza jednak, że taka strona nie może posiadać dynamicznych danych. SSG jest odpowiednie dla widoków statycznych, w których wyświetlane dane się nie zmieniają.

ISR

Incremental Static Regeneration (pol. *inkrementalne odtwarzanie strony statycznej*) to rozwinięcie SSG, które częściowo rozwiązuje problem statycznych danych. ISR pozwala na ponowne wygenerowanie statycznej strony z nowymi danymi, czyli odświeżenie jej. Strona dalej jest w pełni statyczna i jest wysyłany gotowy plik HTML przy żądaniu. Najczęstszym przypadkiem użycia tej strategii są statyczne strony z danymi, które rzadko się zmieniają. Przykładowo, można ustawić ISR w taki sposób, żeby strona odświeżała się raz dziennie z nowymi danymi.

Progressive Hydration

Progressive Hydration (pol. *stopniowe uwadnianie*) to połączenie CSR oraz SSR. Cały statyczny widok strony zostaje wygenerowany na serwerze, natomiast wszystkie komponenty wymagające interaktywności zostają później pobrane przez przeglądarkę w formie skryptu. Statyczna strona zostaje uzupełniona o dynamiczne treści. To rozwiązanie posiada zalety w postaci krótkiego wstępnego załadowania, optymalizacji pod kątem silników wyszukiwarek, stanowi również kompromis pomiędzy generowaniem po stronie serwera oraz przez przeglądarkę.

Streaming SSR

Streaming Server Side Rendering (pol. *strumieniowe renderowanie po stronie serwera*) to podejście, w którym HTML jest wysyłany do klienta stopniowo, w miarę jego generowania przez serwer. W przeciwieństwie do tradycyjnego SSR, serwer nie czeka na wygenerowanie całego dokumentu, tylko przesyła gotowe fragmenty natychmiast. Gdy część strony wymaga odświeżenia, serwer generuje zaktualizowany fragment HTML i wysyła go do przeglądarki bez przeładowania całej strony.

ESR

Edge Side Rendering (pol. *renderowanie krawędziowe*) to technologia pozwalająca na generowanie widoków w infrastrukturze zlokalizowanej geograficznie blisko użytkownika końcowego, w tak zwanym Edge Environment (pol. *środowisko krawędziowe*). Skutkiem takiego działania jest znacząca redukcja opóźnień sieciowych wynikających z odległości geograficznej. Technologie umożliwiające ESR to między innymi AWS Lambda oraz Vercel Edge Functions. Ograniczeniem ESR jest zazwyczaj krótki czas wykonania funkcji oraz ograniczona pamięć dostępna w środowisku brzegowym.

Rozdział 2

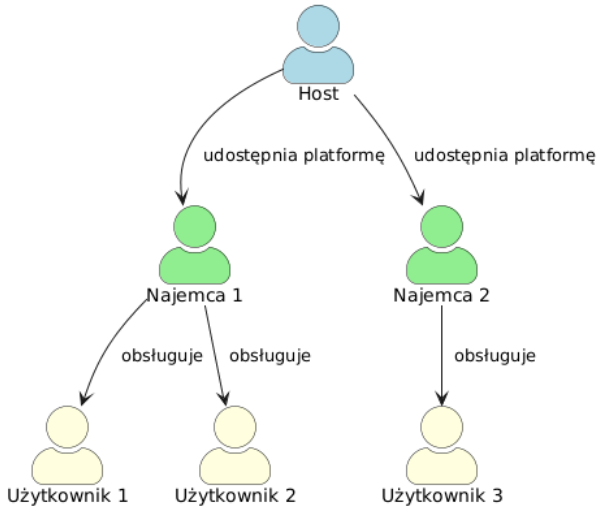
Projektowanie platformy do dystrybucji kursów video

Przed rozpoczęciem implementacji systemu posłużono się zagadnieniami znanymi z inżynierii wymagań [17] oraz przygotowano listę wymagań funkcjonalnych, niefunkcjonalnych, a także przypadki użycia możliwe do zrealizowania w ramach systemu.

Na etapie projektowania wyodrębniono następujące role w systemie:

- Host - właściciel systemu, w tym kreatora platform
- Najemca - osoba posiadająca konto w kreatorze platform oraz tworząca platformy i będąca ich właścicielem
- Użytkownik - osoba posiadająca konto na utworzonej platformie z kursami, klient najemcy.

Pojęcia te występują konsekwentnie w całym systemie. Należy zwrócić uwagę, że pomiędzy Hostem oraz Użytkownikiem nie ma żadnej relacji, co zostało przedstawione na rysunku 6. Jest to zgodne z założeniem opisanym w podrozdziale 1.1.



Rysunek 6. Hierarchia ról w systemie

2.1 Wymagania funkcjonalne i niefunkcjonalne

Personalizacja platformy

Następujące elementy platformy mają być dostosowywalne:

- Paleta kolorów
- Logo
- Strona główna

Paleta kolorów ma składać się z trzech kolorów: głównego, pobocznego oraz koloru tekstu. Kolor główny ma być kolorem reprezentującym markę, której dotyczy platforma. Kolor poboczny powinien być kolorem również kojarzonym z marką lub kolorem dobrze łączącym się z kolorem głównym. Kolor tekstu ma być kolorem używanym w nagłówkach, natomiast nie być kolorem wszystkich tekstów. Kolorem tekstu używanym w większości platformy ma być czarny i ma być niedostosowywalny przez użytkownika. Kreator platformy ma posiadać domyślnie wybraną paletę, którą można w łatwy sposób dostosować.

Najemca ma mieć możliwość przesłania swojego logo, które będzie stale widoczne na utworzonej platformie. Logo ma być plikiem graficznym o rozszerzeniu *png* lub *jpg* oraz ma być przechowywane w systemie. Strona główna ma składać się ze zdjęcia w tle, tekstu głównego oraz dużego przycisku *CTA* (Call To Action). Najemca ma mieć możliwość ustawienia zdjęcia w tle analogicznie do logo platformy oraz zmiany tekstu opisu i przycisku *CTA*. Kolor przycisku ma zostać wygenerowany na podstawie palety. Aplikacja podczas tworzenia platformy ma wyświetlać wizualizację w czasie rzeczywistym, co ma na celu pokazanie najemcy, jak będzie wyglądała jego nowa platforma.

Dodatkowo można rozszerzyć tę listę o następujące, opcjonalne elementy konfiguracyjne:

- Układ kursów
- Widok konkretnego kursu
- Dashboard użytkownika platformy
- Dostępne opcje w odtwarzaczu video
- itd.

Materiały w kursach

Platforma ma wspierać następujące rodzaje materiałów:

- Video
- Tekst
- Quiz

Video to film, który można odtworzyć podczas korzystania z kursu. Platforma ma wspierać możliwość odtwarzania video, w tym celu system musi umożliwiać najemcy przesyłanie oraz zapisywanie plików video. Wspieranym formatem pliku ma być *mp4*. Oprócz tego, system ma automatycznie tworzyć miniaturkę filmu na podstawie pojedynczych klatek video. Opcjonalnie najemca może mieć możliwość samodzielnego wybrania miniaturki z wygenerowanych przez system lub przesłania własnego pliku graficznego. Widok kursu, w którym wybrano materiał video, ma

wyświetlać odtwarzacz video wzorowany na najbardziej popularnych serwisach streamingowych. Musi wspierać następujące opcje:

- Zatrzymywanie oraz wznawianie odtwarzania
- Dostosowywanie głośności
- Tryb pełnoekranowy
- Wybór prędkości odtwarzania
- Automatyczny dobór jakości video, zależny od aktualnej prędkości połączenia

Dodatkowo odtwarzacz może wspierać ręczny wybór jakości video. Kolory odtwarzacza mają dostosowywać się do palety kolorów. System nie ma wspierać strumieniowania video na żywo.

Materiał tekstowy to tekst wspierający formatowanie podobne do *Markdown*. Najemca ma być w stanie w prosty sposób dodawać materiał tekstowy oraz go formatować. Materiał tekstowy nie musi wspierać osadzania obrazów. Musi natomiast wspierać następujące opcje formatowania:

- Wprowadzanie nagłówków
- Pogrubienie
- Kursywa
- Lista numerowana i punktowa

Quiz to sprawdzian, który powinien zostać rozwiązany przez użytkownika kursu. Quiz może zawierać dowolną liczbę pytań, pojedynczego lub wielokrotnego wyboru. Po ukończeniu quizu ma pojawić się informacja o uzyskanym wyniku. Oprócz tego quiz może zostać powtórzony przez użytkownika dowolną liczbę razy. Opcjonalnie, quizy mogą wspierać mechanizm minimalnego progu zdawalności. W tym mechanizmie użytkownik nie może przejść do kolejnego etapu kursu, dopóki nie uzyska pewnego ustalonego wyniku z quizu. Próg zdawalności ma być ustawiany przez najemcę.

Zarządzanie platformami i kursami

Jeden najemca może utworzyć i zarządzać wieloma platformami z poziomu kreatora. Wszystkie platformy muszą być od siebie całkowicie odizolowane pod kątem użytkowników, dostępnych materiałów i kursów.

Najemca ma mieć możliwość dodawania oraz zarządzania kursami w obrębie jednej platformy, której jest właścicielem. System nie może pozwalać na zarządzanie kursami w platformie, która nie należy do danego najemcy. Kurs może składać się z przynajmniej jednego materiału lub być pusty, nie istnieje górny limit materiałów przypiętych do kursu. Jeden materiał nie może być skojarzony z kilkoma kursami. Kolejność materiałów w kursie jest dowolna. Zarządzanie kursami uwzględnia możliwość zmiany nazwy oraz opisu kursu, dodawanie oraz edytowanie treści materiałów, usuwanie materiałów z kursu.

Obsługa platformy przez użytkownika

Użytkownik ma mieć możliwość założenia konta na platformie, ale może również korzystać z platformy jako gość. Niezalogowany użytkownik może mieć dostęp do strony głównej, widoku listy kursów, oraz do wejścia w dany kurs. W widoku kursu gość może przeglądać kurs w trybie podglądu. System ma wyświetlać nazwy oraz rodzaje wszystkich pozostałych materiałów, ale nie zezwalać na przeglądanie ich treści. Ma to pełnić rolę reklamową kursu na platformie, zachęcając gości do rejestracji w celu

uzyskania pełnego dostępu do wszystkich materiałów. Użytkownik może przejść do strony rejestracji z poziomu strony głównej, lub widoku kursu, który właśnie przegląda. Zalogowany użytkownik posiada swój panel sterowania, na którym widzi stan progresu wszystkich swoich rozpoczętych kursów. Po wejściu w dany kurs zalogowany użytkownik może dodać go do „swoich kursów". Ta akcja spowoduje przypisanie kursu do konta użytkownika, w wyniku czego:

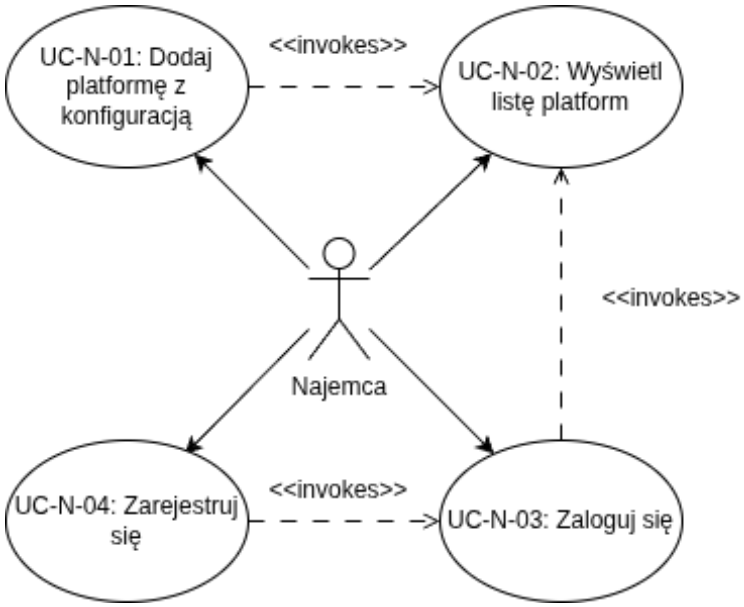
- użytkownik otrzymuje pełen dostęp do materiałów w kursie,
- zostaje zapisany progres użytkownika podczas korzystania z kursu,
- na panelu sterowania użytkownika widnieje zakładka z dodanym kursem oraz wyświetlany jest jego aktualny progres.

2.2 Przypadki użycia

W tym podrozdziale przedstawiono opisy wszystkich przypadków użycia, które zostały wyodrębnione w procesie inżynierii wymagań. Diagram przypadków użycia dla roli Najemcy został rozdzielony na diagramy:

- Najemcy - w przypadku korzystania z systemu z poziomu kreatora platform, rysunek 7,
- Właściciela platformy - w przypadku korzystania z systemu z poziomu utworzonej platformy, rysunek 8.

Przypadki użycia Najemcy



Rysunek 7. Diagram przypadków użycia najemcy

UC-N-01: Dodaj platformę z konfiguracją System pokazuje formularz utworzenia platformy. Najemca uzupełnia konfigurację platformy. System w czasie rzeczywistym prezentuje wizualizację

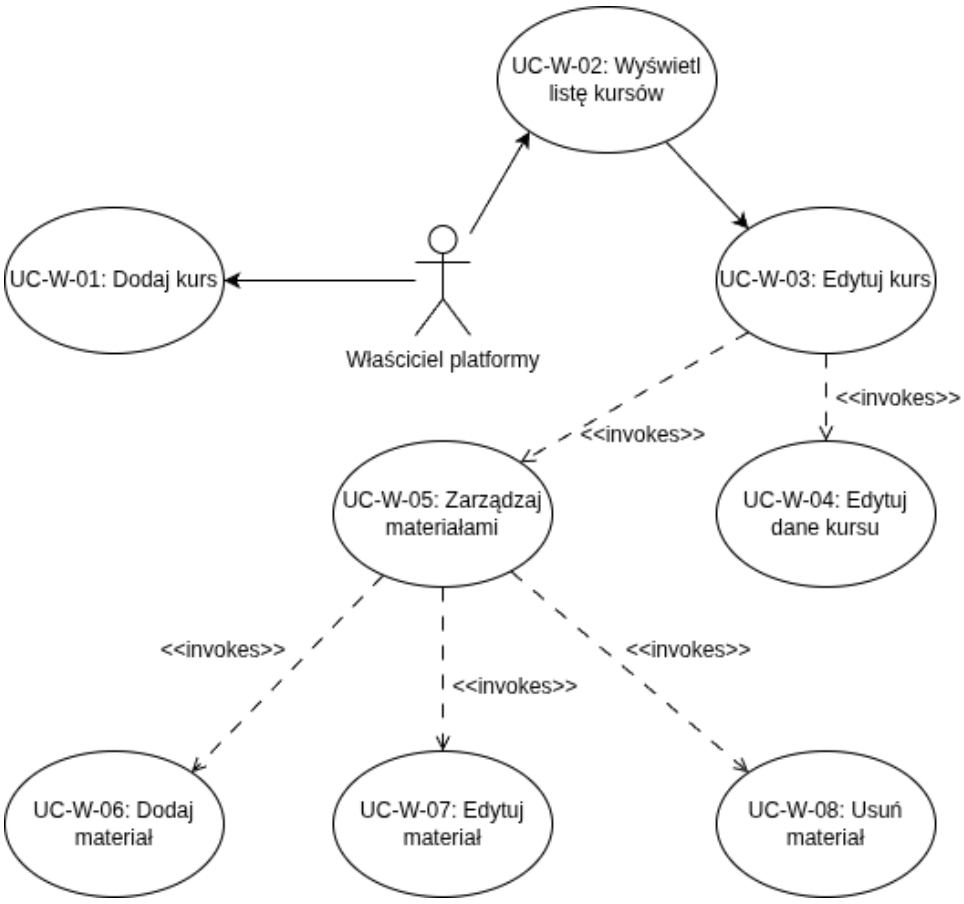
platformy. Po zapisie formularza platforma zostaje utworzona w systemie oraz jest widoczna na liście platform *UC-N-02*.

UC-N-02: Wyświetl listę platform System wyświetla listę platform utworzonych przez Najemcę. Widnieją na niej podstawowe informacje o platformie, takie jak nazwa oraz przycisk do przejścia do platformy.

UC-N-03: Zaloguj się Najemca loguje się w systemie, wykorzystując email i hasło. Po zalogowaniu uruchamia *UC-N-02*.

UC-N-04: Zarejestruj się Najemca wypełnia formularz rejestracji konta w systemie. Po rejestracji uruchamia *UC-N-03*.

Przypadki użycia Właściciela platformy



Rysunek 8. Diagram przypadków użycia właściciela platformy

UC-W-01: Dodaj kurs Właściciel wypełnia formularz i dodaje nowy kurs do swojej platformy. Kurs nie posiada żadnych materiałów w momencie utworzenia.

UC-W-02: Wyświetl listę kursów System wyświetla listę kursów w formie tabeli. W tabeli są widoczne podstawowe dane kursu, oraz przycisk nawigacji do zarządzania kursem *UC-W-03*.

UC-W-03: Edytuj kurs System wyświetla okno, z którego jest możliwe wywołanie przypadków użycia *UC-W-04* oraz *UC-W-05*.

UC-W-04: Edytuj dane kursu System wyświetla formularz z aktualnymi danymi kursu. Najemca edytuje dane oraz wysyła formularz. System zapisuje zaktualizowane dane kursu.

UC-W-05: Zarządzaj materiałami Generyczny przypadek użycia, z którego można uruchomić *UC-W-06*, *UC-W-07* oraz *UC-W-08*.

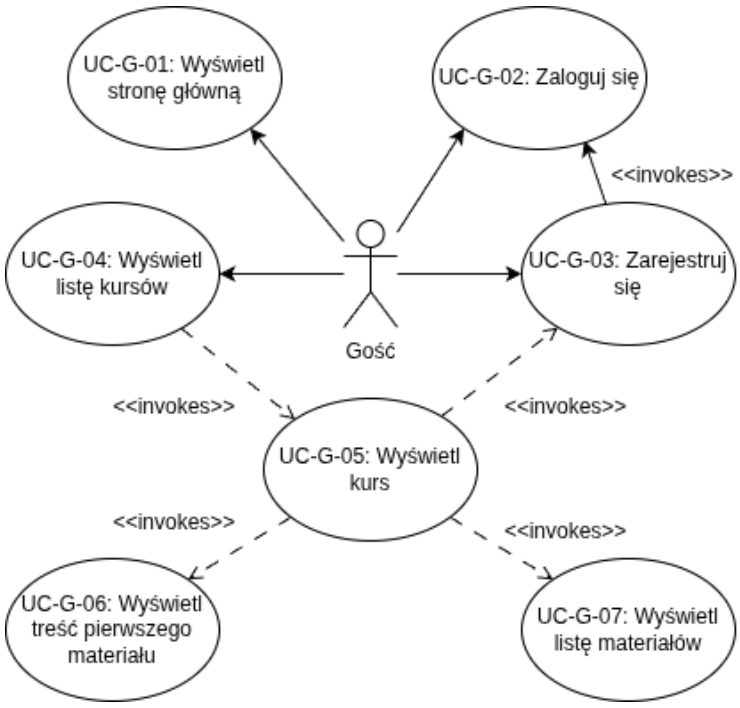
UC-W-06: Dodaj materiał System wyświetla formularz dodawania materiału zależny od wybranego rodzaju materiału:

- Video - System wyświetla formularz danych filmu oraz miejsce do wysłania pliku,
- Materiał tekstowy - System wyświetla edytor tekstowy,
- Quiz - System wyświetla formularz quizu.

UC-W-07: Edytuj materiał System wyświetla formularz edycji materiału, analogiczny do *UC-W-06*. W przypadku filmu można edytować jedynie dane filmu, ale nie plik.

UC-W-08: Usuń materiał System usuwa materiał z kursu. Kolejność materiałów zostaje zaktualizowana.

Przypadki użycia Gościa



Rysunek 9. Diagram przypadków użycia gościa

UC-G-01: Wyświetl stronę główną System wyświetla stronę główną platformy.

UC-G-02: Zaloguj się Gość loguje się na platformie wykorzystując email i hasło. Gość staje się wówczas Użytkownikiem.

UC-G-03: Zarejestruj się Gość wypełnia formularz rejestracji. Po wysłaniu formularza system tworzy nowe konto w platformie oraz uruchamia *UC-G-02*.

UC-G-04: Wyświetl listę kursów System wyświetla listę wszystkich kursów dostępnych na platformie. Po wybraniu kursu uruchamia się *UC-G-05*.

UC-G-05: Wyświetl kurs Generyczny przypadek użycia, z którego można uruchomić *UC-G-03*, *UC-G-06* oraz *UC-G-07*.

UC-G-06: Wyświetl treść pierwszego materiału System wyświetla pierwszy materiał w zależności od rodzaju materiału. Gość nie może wyświetlić kolejnych materiałów.

UC-G-07: Wyświetl listę materiałów System wyświetla listę wszystkich materiałów dostępnych w ramach kursu. Materiały poza pierwszym są dostępne jedynie poglądowo - system wyświetla je w mniej wyrazistych kolorach oraz blokuje interakcję z nimi.

Przypadki użycia użytkownika

UC-U-01: Wyświetl panel sterowania System wyświetla panel sterowania, na którym widnieją aktualne kursy użytkownika oraz ich aktualny progres. Kliknięcie przycisku przy kursie wywołuje *UC-U-02*.

UC-U-02: Kontynuuj kurs Użytkownik zostaje przeniesiony do widoku kursu wyświetlającego ostatnio oglądany materiał.

UC-U-03: Wyświetl listę kursów System wyświetla listę wszystkich kursów dostępnych na platformie.

UC-U-04: Wyświetl kurs System wyświetla aktualny kurs. Jeżeli użytkownik nie posiada kursu w liście swoich kursów, wyświetlany kurs jest prezentowany w trybie podglądu analogicznie do *UC-G-06* oraz można wywołać *UC-U-07*. W zależności od rodzaju materiału uruchamia się *UC-U-06*, *UC-U-10* lub *UC-U-11*.

UC-U-05: Dodaj kurs do moich kursów System zapisuje aktualnie oglądany kurs na liście kursów użytkownika. Ten kurs będzie widoczny w panelu sterowania *UC-U-01*. Po zapisaniu system wyświetla aktualny kurs bez trybu podglądu.

UC-U-06: Wyświetl film System wyświetla odtwarzacz video z aktualnie odtwarzanym filmem. Przypadki użycia związane z odtwarzaczem video to *UC-U-07*, *UC-U-08* oraz *UC-U-09*.

UC-U-07: Zatrzymaj film Użytkownik może wstrzymać oraz wznowić wyświetlanie filmu.

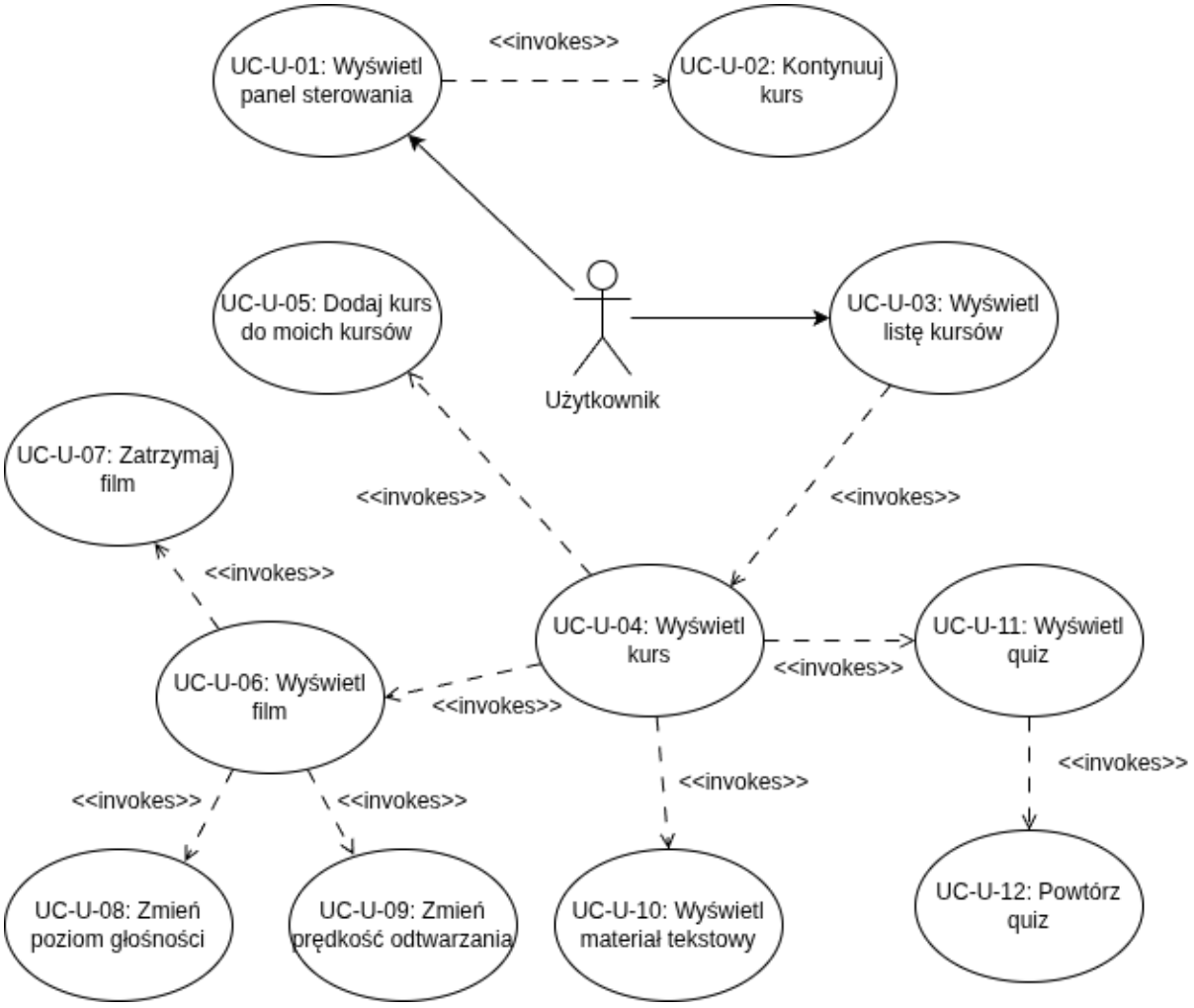
UC-U-08: Zmień poziom głośności Użytkownik może zmniejszyć lub zwiększyć głośność odtwarzanego filmu.

UC-U-09: Zmień prędkość odtwarzania Użytkownik może zmniejszyć lub zwiększyć prędkość odtwarzania filmu.

UC-U-10: Wyświetl materiał tekstowy System wyświetla widok podobny do dokumentu, w którym wyświetla się materiał tekstowy z formatowaniem ustawionym w *UC-W-06*.

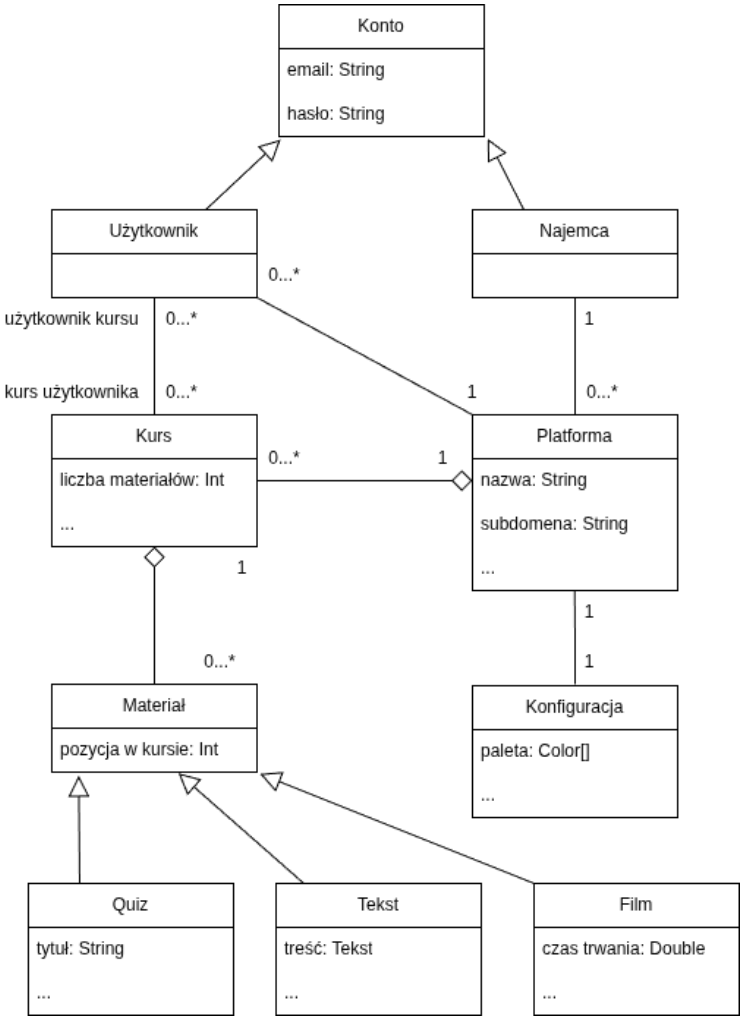
UC-U-11: Wyświetl quiz System wyświetla widok quizu. Użytkownik podaje odpowiedzi, a system pokazuje kolejne pytania. Po wyczerpaniu pytań pokazywany jest wynik quizu.

UC-U-12: Powtórz quiz Po zakończonym quizie Użytkownik może powtórzyć quiz. Aktualny progres zostaje zresetowany, oraz uruchamia się *UC-U-11*.



Rysunek 10. Diagram przypadków użycia użytkownika

2.3 Model dziedziny



Rysunek 11. Model dziedziny

Na rysunku 11 przedstawiono abstrakcyjny model dziedziny. Ma on na celu zaprezentować najważniejsze obiekty znajdujące się w systemie, relacje pomiędzy nimi oraz krotności. Szczegółowe rysunki przedstawiające wszystkie pola encji oraz ich implementację w systemie bazodanowym zostaną zaprezentowane w podrozdziale 3.2.

W aplikacji przewidziano dwa typy osób korzystających z systemu: Użytkownika oraz Najemcę. Obie encje dziedziczą po typie Konto, który zapewnia dane wymagane do logowania w systemie. Rozróżnienie jednego konta na dwie encje ma na celu jedynie zamodelowanie relacji z pozostałymi obiektami. Najemca jest właścicielem zera lub większej liczby platform. Platforma posiada pola takie jak nazwa, opis i przypisana subdomena. Oprócz tego platforma zawiera odniesienie do swojej konfiguracji, która składa się między innymi z wybranej palety kolorów. Użytkownik zawsze jest przypisany do jednej platformy. Jest to zgodne z założeniem biznesowym, wedle którego Użytkownik

tworzy konto na konkretnej platformie, a nie w kreatorze platform. W ramach pojedynczej platformy tworzone są kursy, które Użytkownik może dodać do swojej listy.

Kurs zawiera materiały, których typem może być quiz, tekst lub film. Każdy typ materiału może mieć różne pola. Przykładowo film posiada czas trwania, podczas gdy pozostałe typy nie. Jednakże każdy typ materiału musi posiadać pole określające pozycję danego materiału w kursie. Jeden materiał musi być przypisany do jednego kursu – system nie przewiduje dzielenia jednego materiału pomiędzy wiele kursów. Analogicznie, jeden kurs nie może być przypisany do więcej niż jednej platformy.

2.4 Wybór technologii

Ze względu na złożoność biznesową systemu zdecydowano się zaimplementować backend w frameworku Spring Boot, który jest wiodącym frameworkiem dla aplikacji enterprise w ekosystemie JVM [16][9][19]. Pomimo że ten framework jest najbardziej skojarzony z Javą, wybrano język Kotlin. Kotlin to język programowania oparty na Javie, korzystający z bogatego ekosystemu JVM oraz posiadający dużą liczbę usprawnień z punktu widzenia programisty [6][10]. W społeczności developerów Kotlin jest uznawany jako nowoczesna alternatywa dla Javy oraz słynie z funkcji zwiększających produktywność i czytelność kodu. Jako silnik bazy danych wybrano Postgres, ponieważ jest to stabilne rozwiązanie znane ze swojej wszechstronności i wydajności [12]. Encje w modelu dziedziny nie wymagają elastyczności, dlatego nie wykorzystano żadnej bazy danych NoSQL. Do obsługi plików wykorzystano MinIO, będące technologią do przechowywania obiektów zgodną z API S3, która odpowiada za przechowywanie materiałów video, miniaturek filmów oraz innych plików związanych z konfigurcją platformy [5].

Ponieważ jednym z celów pracy było wykorzystanie hybrydowego renderowania widoków, jako technologię frontendową wybrano Next.js. Jest to framework oparty na React, pozwalający renderować widoki na sposoby opisane w 1.3 [18][1]. Framework ten umożliwia selektywne renderowanie poszczególnych komponentów po stronie serwera lub klienta w ramach tej samej aplikacji, co pozwala na optymalizację zarówno pod kątem wydajności, jak i indeksowania przez wyszukiwarki internetowe. Dodatkowo Next.js wspiera automatyczne dzielenie kodu (code splitting) oraz preładowanie zasobów, co przekłada się na krótszy czas ładowania aplikacji i lepsze doświadczenie użytkownika końcowego.W celu implementacji cachowania konfiguracji platform użyto bazy danych Redis, będącej uznanym rozwiązaniem do realizacji mechanizmów pamięci podręcznej w systemach rozproszonych [2].Przechowywanie często odpytywanych konfiguracji platform w pamięci podręcznej redukuje obciążenie bazy relacyjnej oraz znacząco skraca czas odpowiedzi systemu, co ma szczególne znaczenie w kontekście wielonajemcowego systemu SaaS obsługującego wiele platform jednocześnie.

Za orkiestrację całego środowiska odpowiedzialny jest Minikube, będący dystrybucją Kubernetes uruchamianą lokalnie [4]. Wybór Minikube jako platformy orkiestracyjnej umożliwił pełną symulację środowiska klastrowego na pojedynczej maszynie deweloperskiej, co pozwoliło na testowanie i weryfikację zachowania systemu w warunkach zbliżonych do produkcyjnych bez konieczności

utrzymywania dedykowanej infrastruktury chmurowej. Wszystkie serwisy zostały skonteneryzowane przy pomocy silnika obrazów Docker. Docker stanowi standard konteneryzacji aplikacji, zapewniając szeroką kompatybilność z platformami orkiestracyjnymi oraz rozbudowany ekosystem gotowych obrazów bazowych, co znacząco przyspiesza proces tworzenia i wdrażania mikroserwisów.

Rozdział 3

Implementacja systemu Coursy

Pierwszym krokiem implementacyjnym systemu było zaprojektowanie infrastruktury, która będzie obsługiwała cały system. Ten etap zawierał w sobie między innymi rozplanowanie poszczególnych serwisów oraz schematów baz danych. Następnie zrealizowano mechanizmy, które będą wykorzystywane we wszystkich modułach, jak np. obsługa żetonów JWT. Po implementacji wszystkich serwisów z osobna zadbano o ich orkiestrację w środowisku Kubernetes oraz dodano API Gateway. Ostatnim krokiem było zbudowanie aplikacji Next.js, która obsługuje frontend systemu.

3.1 Podział na mikroserwisy

W wyniku przeprowadzonej inżynierii wymagań zidentyfikowano cztery wyodrębnione konteksty biznesowe, które spełniały kryteria uzasadniające ich separację do autonomicznych mikroserwisów. Były to:

- Platformy z kursami video,
- Kursy,
- Materiały,
- Użytkownicy.

Z modelu dziedziny opracowanego w podrozdziale 2.3 wynika pozornie silna relacja pomiędzy platformami, kursami i materiałami. Natomiast z punktu widzenia implementacji oraz funkcjonalności można je od siebie skutecznie oddzielić. W wielu sytuacjach przypadki użycia dotyczą z osobna tych encji, a nie kilku jednocześnie. Przykładowo, zarządzanie kursem nie wpływa na platformę, w której się znajduje. Tak więc oprócz relacji agregacji, pomiędzy platformą i kursami nie musi istnieć bardziej zaawansowana więź.

Inną sytuacją jest natomiast połączenie pomiędzy kursami i materiałami. Zdarzają się sytuacje, w których przypadek użycia kursów może wpłynąć na materiały w nim zawarte. Przykładowo z przypadku użycia *UC-W-03: Edytuj kurs* można uruchomić *UC-W-05: Zarządzaj materiałami*, który dotyczy materiałów. Z punktu widzenia biznesowego można traktować kurs jako zbiór danych, takich jak nazwa kursu, oraz listę materiałów w nim zawartych. Przyjęto jednak inne założenie, które

pozwoliło na skuteczne rozdzielenie tych obszarów. Jeżeli kurs będzie jedynie zbiorem danych, a dany materiał ma przypisany kurs, to można taką sytuację zamodelować podobnie jak relację platformy i kursu. Fakt, że z przypadku użycia kursu można edytować materiały, nie musi zostać odzwierciedlony architektonicznie. Zarządzanie materiałami można traktować osobno, nie zwracając uwagi na kurs, którego się dotyczy. Takie podejście sprawia, że przypadki użycia dotyczące materiałów stają się niezależne od przypadków użycia kursów.

Kolejnym argumentem, nawet ważniejszym niż logika biznesowa, przemawiającym za rozdzieleniem kursów i materiałów na osobne serwisy, była kwestia wydajnościowa. Obsługa materiałów video stanowi najbardziej obciążającą operację całej aplikacji. Zarówno obsługa plików, jak i przetwarzanie materiałów video (opisane w 3.5) są operacjami wymagającymi znacząco więcej mocy obliczeniowej w porównaniu do pozostałych serwisów, które w większości sytuacji ograniczają się do operacji bazodanowych typu CRUD. Z tego powodu cała obsługa plików została wyniesiona do osobnego mikroserwisu, aby w razie potrzeby móc niezależnie skalować ten komponent systemu. Takie podejście umożliwia zwiększenie liczby replik serwisu materiałów lub przydzielenie mu większych zasobów obliczeniowych (CPU, RAM) bez konieczności skalowania pozostałych serwisów. Taka architektura zapewnia lepsze wykorzystanie zasobów infrastruktury oraz pozwala na elastyczne dostosowanie wydajności systemu do rzeczywistego obciążenia. Opisane rozwiązanie jest również zgodne z założeniami przedstawionymi w podrozdziale 1.2.

Serwis związany z użytkownikami został rozbity na dwa obszary. Pierwszy mikroserwis zawiera logikę biznesową, na którą składa się np. zakładanie kont, role użytkownika, przypisanie do platformy. Drugi serwis obejmuje logikę związaną z autoryzacją użytkowników. Istotne było, aby techniczne zagadnienia związane z żetonami JWT, odświeżaniem i blokowaniem tokenów oraz logowaniem użytkowników w systemie nie znajdowały się w tym samym miejscu co logika zarządzania użytkownikami. Wydzielenie logiki zarządzania autentykacją jest częstym wzorcem wykorzystywanym w architekturze mikroserwisowej w połączeniu z API Gateway.

Rysunek 12 przedstawia finalną architekturę backendu. Przed całą aplikacją został umieszczony API Gateway. Pozostałe odpowiedzialności zostały przydzielone następująco:

- Zarządzanie użytkownikami, rejestracja - *Users*
- Zarządzanie platformami - *Platforms*
- Zarządzanie kursami, przypisanie kursu do użytkownika - *Courses*
- Zarządzanie materiałami, obsługa plików - *Materials*
- Obsługa autentykacji, przechowywanie haseł - *Auth*

3.2 Model danych i strategia bazy danych

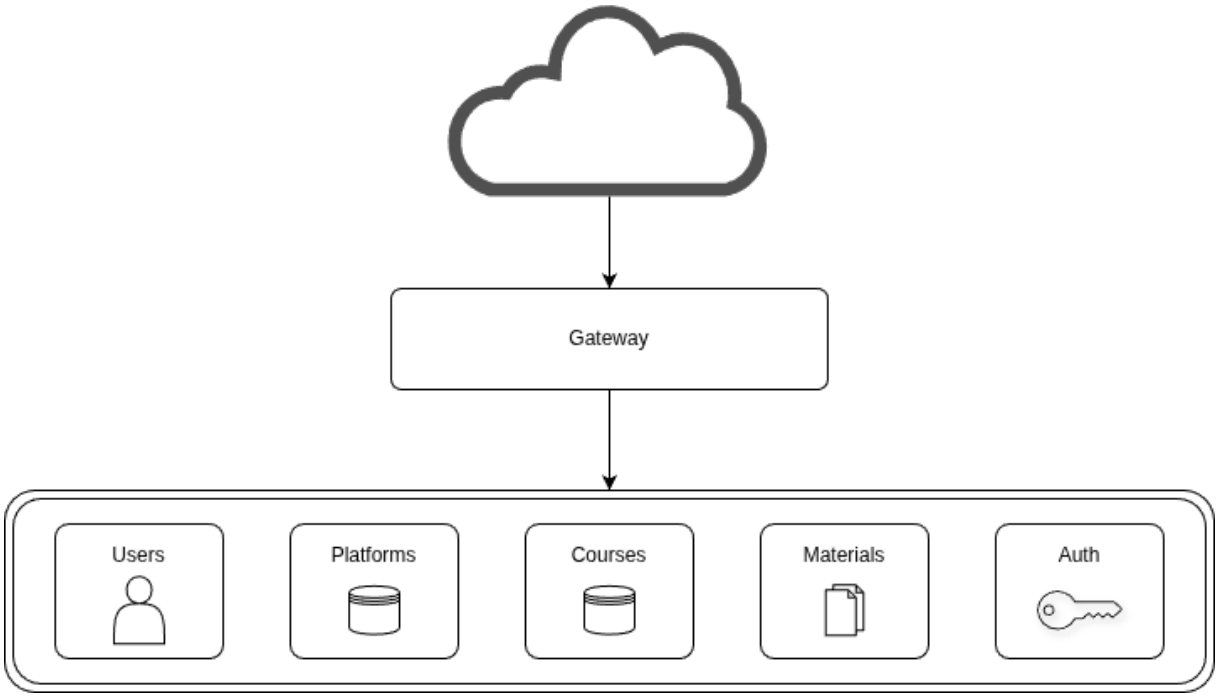
Spośród rozwiązań wspierających wielonajemcowość przedstawionych w podrozdziale 1.1 wybrano podejście z rozróżnianiem danych najemcy w oparciu o kolumnę („Wspólna aplikacja i tabela”). Decyzja ta została podjęta ze względu na to, że pozostałe rozwiązania niosą ze sobą zwiększony narzut implementacyjny, natomiast nie oferują funkcjonalności, które byłyby wykorzystane w systemie.

Model danych zakłada, że dane wszystkich najemców będą miały taką samą strukturę. Nie nastąpi sytuacja, w której jeden najemca będzie musiał przechowywać dane różniące się od pozostałych i wymagających przechowywania w osobnej tabeli lub schemacie. Największą praktyczną zaletą wykorzystania innej strategii byłoby zwiększone bezpieczeństwo danych poszczególnych najemców. W przypadku aplikacji produkcyjnej wypuszczanej na rynek ten argument byłby istotny i mógłby wpłynąć na architekturę bazodanową. Natomiast w kontekście niniejszej pracy, której celem jest zaprezentowanie funkcjonalnego systemu oraz weryfikacja przyjętych rozwiązań architektonicznych, wybrane podejście stanowi optymalne rozwiązanie pod względem relacji kosztu implementacyjnego oraz uzyskanych efektów.

Tabela 1. Przypisanie baz danych do mikroserwisów

Mikroserwis	**Baza danych**
Users	users_db
Platforms	platforms_db
Courses	courses_db
Materials	materials_db
Auth	auth_db

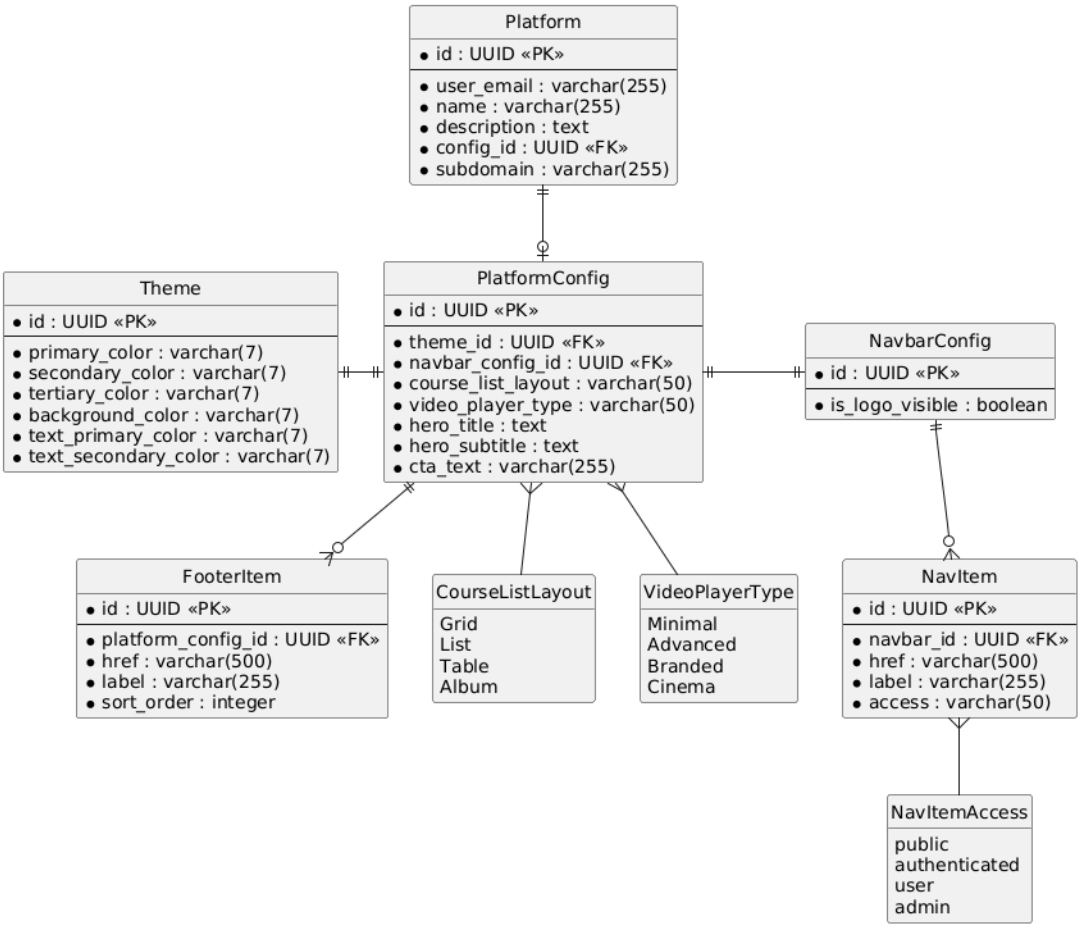
Pomimo tego, że wszyscy najemcy dzielą tabele pomiędzy siebie, to każdy mikroserwis posiada swoją własną bazę danych. Mikroserwisy nie dzielą się bazą danych, dzięki temu są bardziej niezależne. Wiąże się z tym jednak kilka konsekwencji implementacyjnych. Po pierwsze, nie można wykorzystać relacji klucza obcego pomiędzy tabelami różnych mikroserwisów. Przykładowo, podczas tworzenia



Rysunek 12. Podział na mikroserwisy

nowego kursu w serwisie *Courses* należy zapisać odpowiedni *platform_id*, jednak nie ma możliwości zdefiniowania referencyjnej więzi z tabelą *Platform* znajdującą się w bazie danych *platforms_db*. Walidacja poprawności danych musi zatem odbywać się na poziomie logiki aplikacyjnej. Po drugie, w sytuacji, gdy endpoint wymaga danych z innego mikroserwisu, konieczna jest komunikacja międzyserwisowa poprzez API. Gdyby wszystkie serwisy korzystały ze wspólnej bazy danych lub schematu, można by bezpośrednio odpytywać tabele z innych obszarów. Takie podejście byłoby jednak sprzeczne z filozofią mikroserwisów, która zakłada luźne powiązanie i autonomię poszczególnych komponentów systemu.

Platforms



Rysunek 13. Baza danych platforms_db

Na rysunku 13 zaprezentowano strukturę bazy danych *platforms_db*. Najważniejszą tabelą jest *Platform* i reprezentuje platformę utworzoną przez Najemcę. Posiada odniesienie do Najemcy poprzez *Platform.user_email*, który w tym przypadku jest identyfikatorem użytkownika. Pole *Platform.id* jest szczególnie istotne, ponieważ będzie ono używane w każdym zasobie obecnym w platformie. Jest to wartość, która przypisuje dany zasób do odpowiedniej platformy Najemcy. Użycie *Platform.subdomain* zostanie wytłumaczone w podrozdziale 3.4.

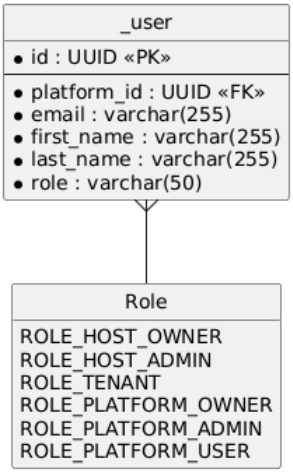
Tabela *PlatformConfig* reprezentuje konfigurację danej platformy. Struktura bazy danych posiada wiele dodatkowych pól i tabel pozwalających na konfigurację wizualną platformy na więcej opcji, niż przewidziano w projekcie systemu. Nie są jednak aktualnie wspierane przez frontend. W przypadku tworzenia platformy opcje konfiguracyjne poza paletą kolorów w tabeli *Theme* otrzymują domyślne wartości. Przyjęta struktura bazy danych umożliwia rozszerzenie funkcjonalności systemu w przyszłości.

CourseListLayout to enum przedstawiający tryb wyświetlania listy kursów. Aktualnie wspieranym trybem wyświetlania jest Grid. *VideoPlayerType* to enum przedstawiający tryb wyświetlacza kursów video. Aktualnie wspieranym trybem jest Minimal. Z biznesowego punktu widzenia można udostępniać bardziej rozbudowane opcje konfiguracyjne Najemcom, którzy korzystają z rozszerzonych planów subskrypcyjnych.

System może wspierać dostosowywalne elementy nawigacji w Navbar oraz w Footerze strony. *NavItemAccess* to enum wyrażający poziom dostępu i wyświetlanie danego elementu użytkownikowi. Przykładowo, element Navbaru z oznaczeniem *NavItemAccess.admin* pokaże się tylko zalogowanemu administratorowi (Najemcy). W aktualnej implementacji platforma otrzymuje przypisany domyślną konfigurację Navbara oraz Footera.

Tabela *Theme* jest odpowiedzialna za przechowywanie palety kolorów skojarzonej z platformą. Oprócz pól wymaganych w projekcie znajdują się w niej również dodatkowe pola z kolorami, na przykład *background_color*. Podobnie do poprzednich cech konfiguracyjnych, nie są aktualnie obsługiwane przez frontend.

Users



Rysunek 14. Baza danych users_db

Rysunek 14 przedstawia bazę danych mikroserwisu *Users*. Składa się ona tylko z jednej tabeli zawierającej dane biznesowe użytkownika oraz jego rolę w systemie. Tabela nazywa się *_user*, ponieważ *User* jest zarezerwowanym słowem w Postgres.

Pole *_user.platform_id* informuje system, do jakiej platformy należy użytkownik. Jeżeli znajduje się tam wartość NULL, oznacza to, że użytkownik posiada konto w kreatorze platform, a nie w platformie. Należy zwrócić uwagę na to, że baza danych przewiduje więcej ról, niż jest aktualnie używanych w systemie:

- *ROLE_HOST_OWNER* - Właściciel systemu Coursy
- *ROLE_HOST_ADMIN* - Administrator systemu
- *ROLE_TENANT* - Konto najemcy w systemie (kreatorze platform)
- *ROLE_PLATFORM_OWNER* - Konto właściciela platformy
- *ROLE_PLATFORM_ADMIN* - Administrator platformy
- *ROLE_PLATFORM_USER* - Użytkownik platformy

Najemca posiadający konto w kreatorze platform posiada *_user.platform_id* ustawione na NULL. W momencie utworzenia platformy system tworzy nowe konto dla Najemcy z rolą ustawioną jako *ROLE_PLATFORM_OWNER*, takimi samymi danymi użytkownika oraz odpowiednim *_user.platform_id*. Dzięki temu Najemca może zalogować się w swojej platformie oraz posiada odpowiednie uprawnienia. Przykład takiej sytuacji został przedstawiony w tabeli 2. W przypadku gdyby system nie tworzył nowego konta na nowej platformie oraz pozwalał najemcy logować się kontem w kreatorze platform, mogłoby dojść do sytuacji, w której dowolny najemca może zalogować się na dowolnej platformie i potencjalnie mieć w niej uprawnienia właściciela.

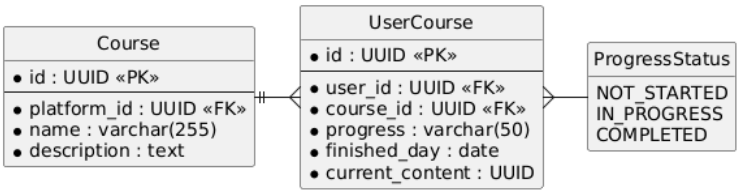
Tabela 2. Przykład kont najemcy w systemie

platform_id	**firstname**	**rola**	**komentarz**
NULL	Najemca 1	ROLE_TENANT	Konto najemcy w kreatorze platform
123	Najemca 1	ROLE_PLATFORM_OWNER	Konto właściciela na platformie

Baza danych oraz mikroserwis wspierają również obsługę administratorów kreatora kursów oraz platform. Jednakże frontend aktualnie nie posiada zaimplementowanego modułu zarządzania użytkownikami, w tym tworzenia administratorów, więc jest to kolejne miejsce, w którym można potencjalnie rozwinąć system.

Tabela *_user* przechowuje jednocześnie dane Najemców oraz Użytkowników. Ma to swoje odzwierciedlenie w modelu dziedziny 2.3, według którego Najemca oraz Użytkownik dziedziczą po encji Konto, nie posiadają innych pól, a różnią się jedynie asocjacjami. W tej bazie danych nie znajdują się jednak encje, z którymi Najemca bądź Użytkownik mieliby wchodzić w relację, wobec czego ich reprezentacja bazodanowa może zostać zaimplementowana jako jedna tabela.

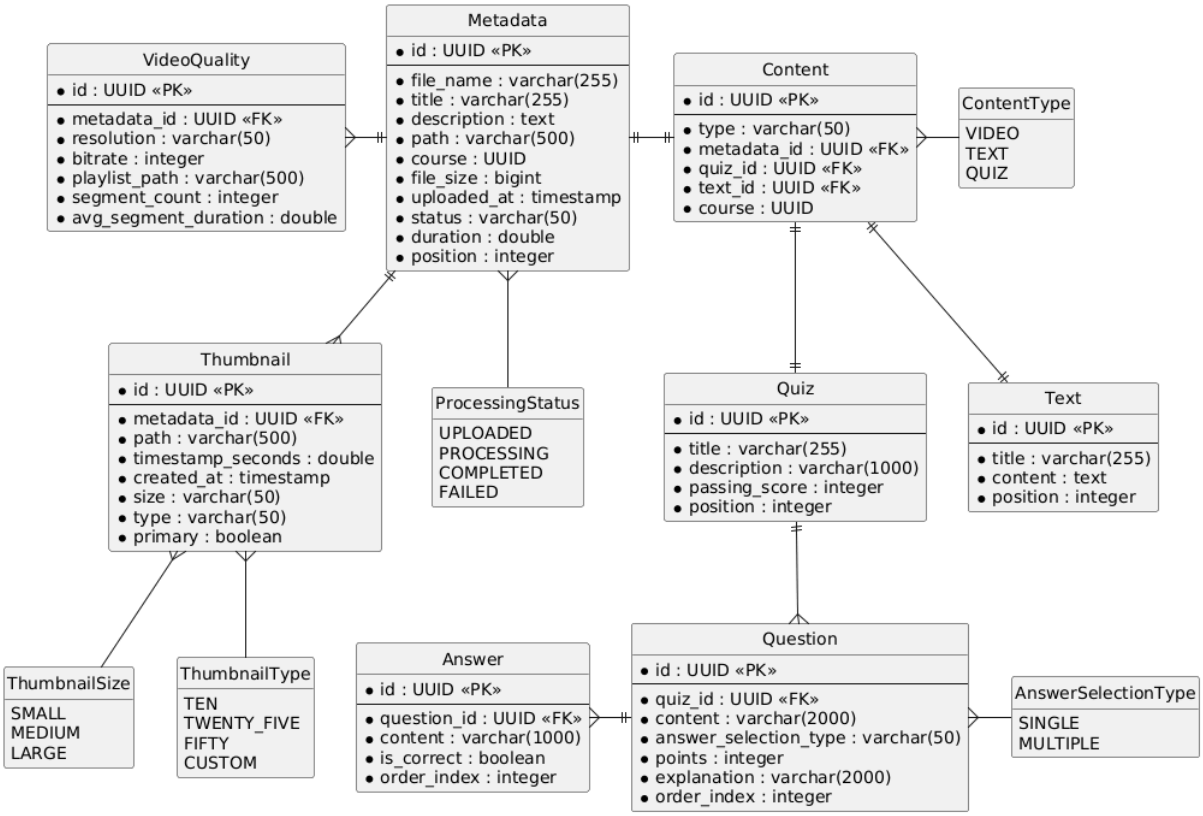
Courses



Rysunek 15. Baza danych courses_db

Rysunek 15 przedstawia schemat bazy danych mikroserwisu *Courses*. Składa się z dwóch tabel. Tabela *UserCourse* reprezentuje kurs, który został przypisany do konta Użytkownika. Progres w kursie zostaje zapisany w dwóch polach: *UserCourse.current_content* posiada odwołanie do ostatniego oglądanego materiału, natomiast *UserCourse.progress* jest oznaczeniem, czy kurs jest nierozpoczęty, w trakcie lub ukończony.

Materials



Rysunek 16. Baza danych materials_db

Na rysunku 16 została przedstawiona struktura bazy danych mikroserwisu *Materials*. Tabele po lewej stronie rysunku, to jest *Metadata* oraz tabele z nią połączone, zostaną szczegółowo omówione w rozdziale 3.5, ponieważ dotyczą obsługi materiałów video.

Tabele *Metadata*, *Quiz* oraz *Text* reprezentują materiały dostępne w ramach kursu. Te encje dziedziczą po typie *Content*, który posiada odniesienie do kursu. *Content* posiada ustawione tylko jedno z trzech pól odnoszących się do szczegółów materiału. Dodatkowo pole *Content.type* jest oznaczeniem jakiego typu materiału jest dany materiał. Przykładowo, jeżeli w kursie dodany jest materiał tekstowy, to *Content* posiada ustawiony typ na *ContentType.TEXT* oraz *text_id* odnoszący się do zawartości tego materiału. Pozostałe *metadata_id* oraz *quiz_id* są wówczas ustawione na NULL. Każdy materiał posiada pole *position* informujące o kolejności danego materiału w kursie.

Tabele *Quiz* oraz *Question* przechowują informacje dotyczące quizów oraz pytań. Zgodnie z wymaganiami pytanie w quizie może być jednokrotnego lub wielokrotnego wyboru. W bazie danych zostały dodatkowo zaimplementowane pola dotyczące punktacji danego pytania oraz minimalny próg zdawalności kursu, opisany w 2.1. Ta funkcjonalność nie została zaimplementowana po stronie frontendowej.

Baza danych *auth_db* zostanie opisana w następnym podrozdziale 3.3 ze względu na swoje znaczenie techniczne, a nie biznesowe.

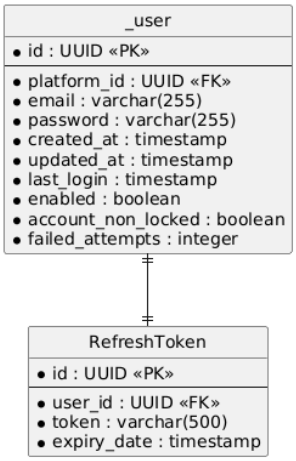
3.3 System autentykacji i autoryzacji

Autentykacja i autoryzacja w aplikacji zostały oparte o żetony JWT w połączeniu z refresh tokenami oraz blacklistą tokenów po wylogowaniu użytkownika [11]. JWT zostaje podpisany tajnym kluczem HMAC256 oraz jest przechowywany na frontendzie jako cookie. Pojedynczy żeton JWT w systemie składa się z następujących elementów:

- **subject** – adres email konta,
- **id** – UUID konta,
- **platformId** – identyfikator platformy konta (może być NULL w przypadku konta utworzonego w kreatorze platform),
- **role** – nazwa roli użytkownika w systemie,
- **iat** (issued at) – timestamp momentu wygenerowania tokenu,
- **exp** (expiration time) – timestamp wygaśnięcia tokenu, obliczany jako czas utworzenia wraz z dodanym skonfigurowanym czasem ważności (domyślnie 7 dni).

Proces tworzenia i zarządzania tokenami, w tym blokowaniu ich na blackliście, został zaimplementowany w serwisie *Auth*. Znajduje się w nim również przechowywanie funkcji skrótu haseł kont użytkowników. Jednakże funkcjonalność walidacji przychodzących tokenów JWT została przeniesiona do API Gateway. Jest to spowodowane tym, że gdyby walidacja żetonów odbywała się w serwisie *Auth*, to każde żądanie przychodzące do aplikacji musiałoby zostać domyślnie przekierowane do tego serwisu. Spowodowałoby to znaczące zwiększenie narzutu związanego z komunikacją międzyserwisową, zwłaszcza że walidacja żetonu tyczy się każdego autoryzowanego żądania. W związku z tym sama walidacja żetonu odbywa się już w warstwie API Gateway. Jeżeli żeton jest nieprawidłowy, to API Gateway zwraca kod błędu, a żądanie nie trafia dalej do aplikacji. Jeżeli

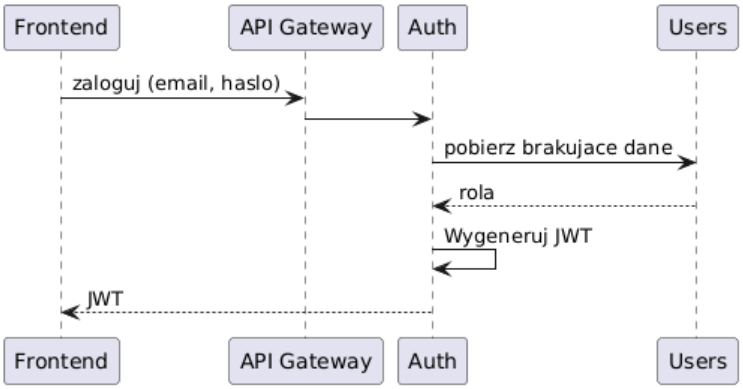
JWT przejdzie etap walidacji, to żądanie trafia wgłąb systemu, a pojedyncze serwisy nie sprawdzają już jego poprawności. Zakładają, że na tym etapie żeton JWT musi być już sprawdzony.



Rysunek 17. Baza danych auth_db

Na rysunku 17 przedstawiono strukturę bazy danych serwisu *Auth*. Tabela *_user* przechowuje techniczne dane dotyczące konta w systemie i różni się od serwisu *Users* tym, że dotyczy ona tylko kwestii związanych z logowaniem konta. Przykładowo, nazwa użytkownika jako pole biznesowe jest zaimplementowane w mikroserwisie *Users*, natomiast hasło lub data ostatniego logowania znajduje się w mikroserwisie *Auth*.

Ponieważ żeton JWT posiada pole dotyczące roli, które nie jest przechowywane w serwisie *Auth*, to proces generowania nowego tokenu JWT musi uwzględniać komunikację z *Users* w celu pobrania brakujących danych biznesowych. Proces ten przedstawiono na rysunku 18.



Rysunek 18. Proces logowania użytkownika w systemie

Po otrzymaniu żetonu JWT w żądaniu serwis docelowy lokalnie ustala reguły autoryzacyjne. Oznacza to, że każdy mikroserwis samodzielnie ustala uprawnienia, które posiada dana rola. Alternatywnym rozwiązaniem mogłoby być obsługiwanie autoryzacji w serwisie *Auth*, ale wtedy cała autoryzacja musiałaby się odbywać globalnie. Zmiana reguły autoryzacyjnej w mikroserwisie

musiałaby wiązać się ze zmianą reguł również w *Auth*. Dodatkowo niektóre reguły autoryzacyjne opierają się na większej liczbie informacji niż tylko rola w systemie. Przykładowo, dodanie kursu do platformy wymaga nie tylko roli właściciela platformy, ale również przypisanego odpowiedniego *platform_id* do konta. W innym przypadku dowolny właściciel mógłby dodawać kurs w dowolnej platformie tylko dlatego, że posiada odpowiednią rolę. Takie globalne zarządzanie autoryzacją przynosi niewspółmierną ilość narzutu implementacyjnego w stosunku do potencjalnych korzyści. Oprócz tego serwis *Auth* musiałby znać szczegóły implementacyjne innych serwisów, co psuje koncepcję mikroserwisów.

3.4 Identyfikacja najemcy i routing żądań

Na maszynie z uruchomioną aplikacją został skonfigurowany lokalny serwer DNS, który przekierowuje wszystkie żądania z postfixem **.coursy.com* do punktu wejściowego aplikacji. Najemca podczas tworzenia platformy wybiera również domenę, pod którą będzie ona dostępna. Przykładowo, platforma z subdomeną *edukorp* będzie dostępna pod adresem *edukorp.coursy.com*. W momencie pierwszego wejścia na stronę frontend wysyła zapytanie do *Platforms*, które można opisać „Jakie *platform_id* posiada platforma z subdomeną *edukorp*?". Po uzyskaniu tej informacji *platform_id* zostaje zapisane w cookies. Jeżeli dane żądanie wymaga kontekstu platformy, to zostaje ono pobrane z cookies oraz dołączone do ciała żądania. Innym rozwiązaniem mogłoby być dodawanie id platformy do każdego żądania wychodzącego z aplikacji, na przykład w postaci nagłówka *X-PLATFORM-ID*. Zaletą tego podejścia mogłaby być uproszczona konfiguracja klienta API po stronie frontendu, ponieważ nie trzeba by ręcznie pobierać id oraz ustawiać go w żądaniu. Niemniej jednak obydwa rozwiązania można traktować jako odpowiednie.

Należy podkreślić, że opisana konfiguracja z lokalnym serwerem DNS jest rozwiązaniem demonstracyjnym, stosowanym wyłącznie w środowisku deweloperskim. We wdrożeniu produkcyjnym konieczne byłoby skonfigurowanie rzeczywistych rekordów DNS, certyfikatów SSL oraz odpowiedniej infrastruktury sieciowej.

3.5 Streaming video z użyciem protokołu HLS

Do strumieniowania materiałów video zaimplementowano API współpracujące z protokołem *HTTP Live Streaming*. Protokół ten wprowadza następujące założenia:

- Każde video zostaje podzielone na fragmenty o długości 6-10 sekund,
- Te same segmenty zostają wyrenderowane w wielu rozdzielczościach,
- Wszystkie segmenty w danej rozdzielczości zostają opisane plikiem *Quality playlist*,
- *Master playlist* wskazuje na wszystkie dostępne playlisty.

Proces strumieniowania pliku do odtwarzacza:

1. Przeglądarka pyta się backend o *Master playlist* dla danego filmu,
2. Backend wysyła *Master playlist*,
3. Przeglądarka w oparciu o *Master playlist*, konfigurację odtwarzacza oraz dostępnej prędkości łącza pobiera z backendu wybiera jakość oraz pobiera *Quality playlist*,
4. Backend wysyła odpowiedni *Quality playlist*,
5. Przeglądarka wie na podstawie *Quality playlist*, z ilu segmentów składa się materiał. Rozpoczyna pobieranie pierwszego segmentu.
6. W międzyczasie przeglądarka pobiera i buforuje kolejne segmenty materiału.

Przeglądarka może przełączyć się na inną jakość w trakcie odtwarzania filmu. W tym celu musi pobrać nowy *Quality playlist*. Ponieważ wszystkie segmenty trwają dokładnie tyle samo, to nic nie stoi na przeszkodzie, żeby pobrać pierwsze dwa segmenty w rozdzielczości 480p, a następnie przełączyć się na 720p i pobierać dalsze segmenty w lepszej rozdzielczości. Listingi 1 oraz 2 przedstawiają przykładowe pliki *Master playlist* i *Quality playlist* utworzone dla 30-sekundowego materiału video.

W serwisie *Materials* zostały zaimplementowane odpowiednie punkty końcowe wspierające strumieniowanie video:

GET /api/content/videos/{video_id}/master.m3u8 Pobieranie *Master playlist*

GET /api/content/videos/{video_id}/{quality}/playlist.m3u8 Pobieranie *Quality playlist*

GET /api/content/videos/{video_id}/{quality}/{segment_name} Pobieranie odpowiedniego segmentu w zależności od jakości.

Dostępne jakości to 480p, 720p, 1080p. Do przechowywania plików wymaganych do użycia protokołu HLS został wykorzystany MinIO. Pliki zostają zapisane w buckecie *Videos* w następujących ścieżkach, analogicznych do API:

videos/{course_id}/{video_id}/{file_name}.mp4 Oryginalny przesłany plik

videos/{course_id}/{video_id}/master.m3u8 *Master playlist*

videos/{course_id}/{video_id}/{quality}/playlist.m3u8 *Quality playlist*

videos/{course_id}/{video_id}/{quality}/{segment_name} Pojedynczy segment

videos/{course_id}/{video_id}/thumbnails/ Folder z wygenerowanymi miniaturkami filmu

```
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-INDEPENDENT-SEGMENTS
#EXT-X-STREAM-INF:BANDWIDTH=800000,RESOLUTION=854x480
480p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=1400000,RESOLUTION=1280x720
720p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=2800000,RESOLUTION=1920x1080
1080p/playlist.m3u8
```

Listing 1. master.m3u8

```
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-TARGETDURATION:8
#EXT-X-MEDIA-SEQUENCE:0
#EXT-X-PLAYLIST-TYPE:VOD
#EXTINF:8.333333,
segment_000.ts
#EXTINF:8.333333,
segment_001.ts
#EXTINF:8.333333,
segment_002.ts
#EXTINF:5.033333,
segment_003.ts
#EXT-X-ENDLIST
```

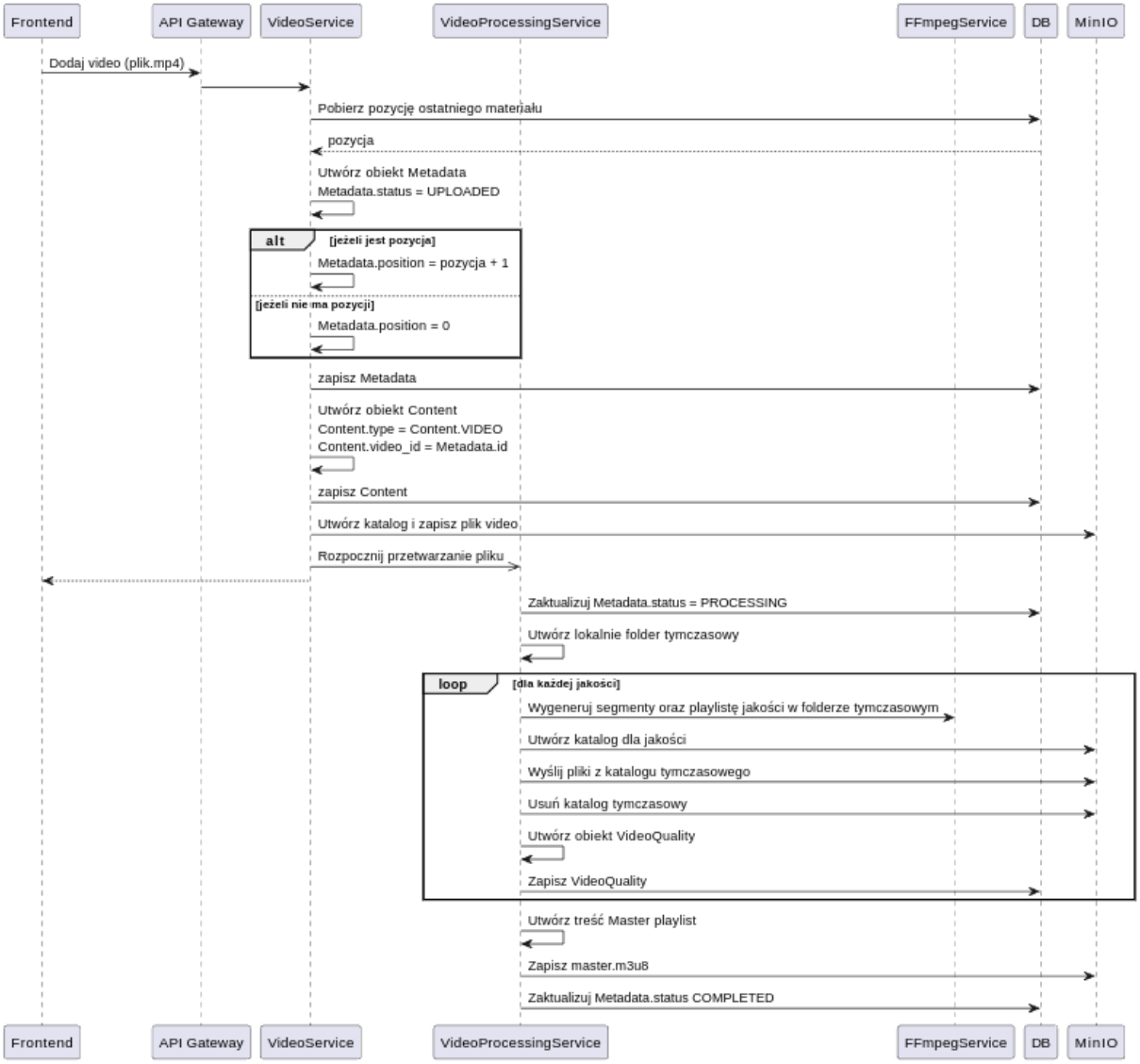
Listing 2. playlist.m3u8

W podrozdziale 3.2 na rysunku 13 został przedstawiony schemat bazy danych odpowiadający za przechowywanie metadanych materiałów video. Tabela *Metadata* zawiera informacje dodatkowe o filmie, takie jak tytuł oraz opis. Posiada relację z *VideoQuality*, która prezentuje dostępne rozdzielczości filmu. Nie wolno zakładać, że każdy materiał posiada wszystkie 3 jakości. Przykładowo, materiał w oryginalnej rozdzielczości 480p nie zostanie sztucznie wygenerowany w rozdzielczości 1080p. Tabela *Thumbnail* zawiera metadane miniaturek. System generuje 3 miniaturki, korzystając z klatek video w 10%, 25% oraz 50% materiału. Oprócz tego generuje je w 3 rozdzielczościach. Daje to łącznie 9 kombinacji. Dodatkowo użytkownik może również wysłać swoją własną miniaturkę, oznaczaną w systemie jako *ThumbnailType.CUSTOM*. Ta funkcjonalność nie jest jednak obsługiwana na frontendzie.

Do przetwarzania plików *.mp4* do wymaganego formatu HLS po stronie serwera wykorzystano oprogramowanie *FFmpeg*[1]. Jest ono odpowiedzialne za utworzenie plików z playlistami oraz pojedynczych segmentów. Dodatkowo *FFmpeg* jest używany do tworzenia obrazów na podstawie poszczególnych klatek video, które zostają następnie wykorzystane jako miniaturki filmu. Rysunek

[1] https://www.ffmpeg.org/

19 zawiera diagram sekwencji prezentujący proces przetwarzania materiału. Jest to najbardziej zaawansowana operacja zaimplementowana w systemie.



Rysunek 19. Proces przetwarzania pliku video

W celu wyświetlania video frontend korzysta z komponentu biblioteki *react-player*[2], który posiada wbudowaną obsługę protokołu HLS. Komponent został skonfigurowany w taki sposób, żeby jego wygląd pasował do konfiguracji platformy. Dodatkowo odtwarzacz video wyświetla miniaturkę filmu, zanim zostanie uruchomiony.

[2]https://github.com/cookpete/react-player

3.6 Obsługa warstwy frontend

Frontend aplikacji został zbudowany w frameworku Next.js, który wspiera hybrydowe renderowanie widoków. Pojedyncze strony składają się z mieszanki komponentów klienckich oraz serwerowych. Celem implementacyjnym w systemie było renderowanie jak największej ilości widoków po stronie serwera, aby zoptymalizować aplikację pod kątem SEO. Wszystkie strony, poza formularzami rejestracji i logowania, składają się z trzech następujących elementów:

- Navbar - menu nawigacyjne na górze ekranu,
- Body - treść widoku,
- Footer - stopka na dole ekranu.

Rysunek 20 prezentuje wygląd typowej strony dostępnej w aplikacji. Elementy Footer oraz Navbar są renderowane po stronie serwera, ponieważ nie obsługują dynamiczności. W tym przypadku również widok strony głównej udało się wygenerować całkowicie po stronie serwera, ponieważ przycisk „Sprawdź teraz" nie posiada logiki i jest to przestylowany znacznik *<a>*. Gdyby był to *<button>*, to możliwe, że ten widok należałoby renderować po stronie przeglądarki. Nie wszystkie widoki jednak udało się zaimplementować po stronie serwera. Są to głównie formularze, które wywołują walidację lub żądania do backendu. Jednakże nawet w tych przypadkach część kliencka widoku zostawała minimalizowana po to, żeby jak najmniej UI musiało renderować się po stronie przeglądarki.

Rysunek 20. Strona główna wygenerowanej platformy

Każda strona musi mieć dostęp do konfiguracji platformy. W tym celu wszystkie strony zostały opakowane w globalny *RootLayout*, którego celem jest pobieranie danej konfiguracji platformy i serwowanie jej do wszystkich komponentów. *RootLayout* jest renderowany po stronie serwera. Ponieważ *RootLayout* uruchamia pobieranie konfiguracji na każdej stronie, mogłoby to być zbędne obciążenie dla backendu. Aby temu zapobiec, zaimplementowano warstwę cache przy pomocy *Redis*,

której celem jest przechowywanie już raz pobranej konfiguracji. Frontend przy każdym żądaniu sprawdza, czy w pamięci podręcznej jest przechowywana konfiguracja wymaganej platformy. Jeżeli tak, to wykorzystuje ją. Jeżeli nie, to pobiera świeże dane z backendu oraz wynik zapisuje w cache. Dzięki temu, jeżeli stronę platformy odwiedzi 10 nowych użytkowników, to frontend wykona tylko jedno żądanie do backendu, a następnie będzie serwował pozostałym użytkownikom konfigurację zapisaną w cache.

```
function CustomizedTitle() {
  // Pobranie konfiguracji platformy z backendu lub z cache
  const config = await getCachedConfig();

  // Wykorzystanie w UI
  return <h1>{config.title}</h1>;
}
```

Listing 3. Używanie konfiguracji w komponencie

Warto zwrócić uwagę na to, że jeden frontend jest odpowiedzialny nie tylko za renderowanie widoków wszystkich platform, ale jednocześnie kreatora kursów. Jeżeli zostanie użyty URL *coursy.com* bez żadnej subdomeny, to *platform_id* zostaje ustawiony na NULL, a frontend działa wówczas w trybie kreatora platform. Zostają wówczas używane widoki i strony dostosowane pod kreator platform, a strony takie jak *Moje kursy* stają się zablokowane. Kreator platform również posiada swoją konfigurację, z której korzysta, oraz jest zapisany na backendzie jako platforma z *platform_id* ustawionym na NULL.

3.7 Orkiestracja

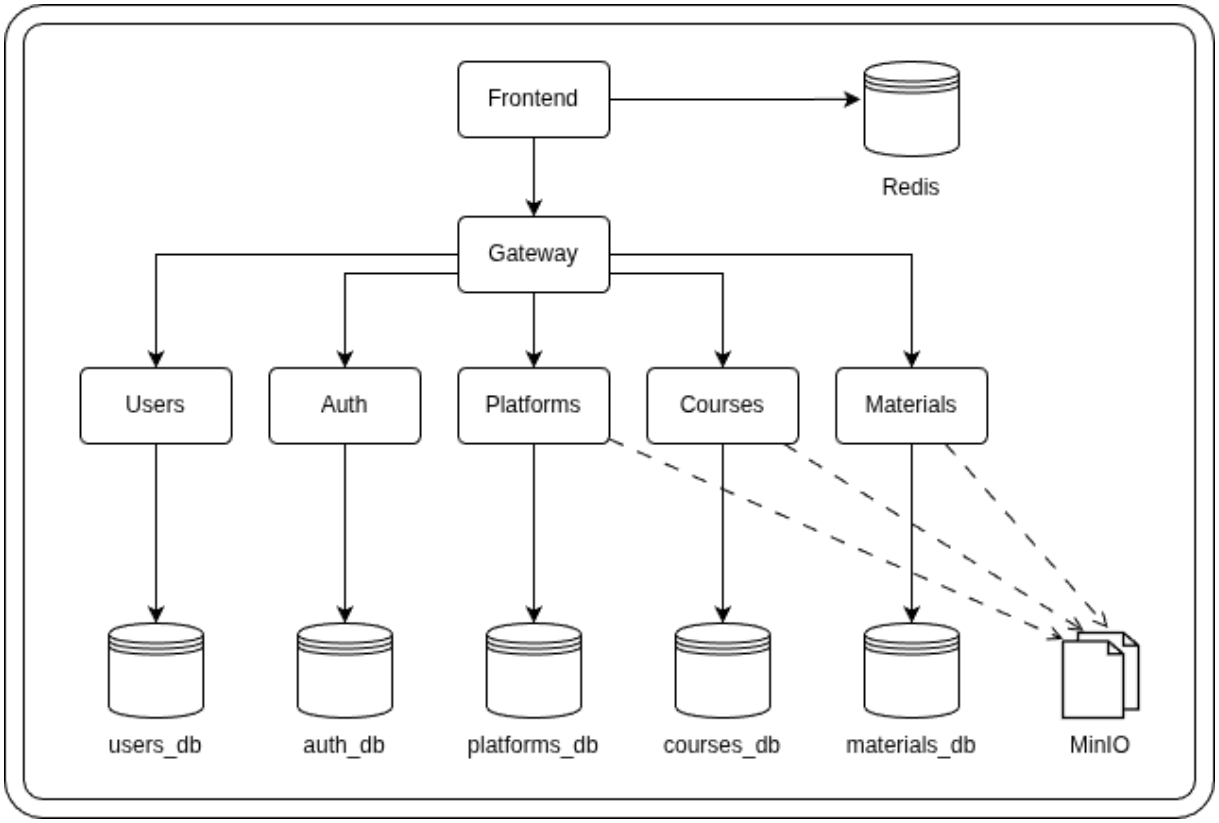
Rysunek 21 przedstawia zaimplementowaną architekturę Kubernetes, która obsługuje orkiestrację systemu Coursy. Zgodnie z implementacją przedstawioną w 3.2, każdy serwis posiada własną bazę danych. Oprócz mikroserwisów oraz baz danych została utworzona usługa przechowywania plików MinIO, z której korzystają jednocześnie 3 serwisy:

- *Platforms* - przechowuje logo oraz zdjęcie w tle strony głównej danej platformy,
- *Courses* - przechowuje miniaturki kursów,
- *Materials* - przechowuje pliki związane z materiałami video.

Każdy serwis przechowuje pliki w osobnym buckecie. Teoretycznie można by utworzyć usługę MinIO dla każdego serwisu z osobna, ale nie niesie to żadnych znaczących korzyści. Zaletą zwiększenia ilości MinIO byłoby zwiększenie bezpieczeństwa, jednakże pliki przechowywane w MinIO są w większości dostępne publicznie, zwłaszcza zdjęcia obecne w *Platforms* oraz *Courses*. Jedynymi zasobami, które nie są dostępne dla każdego, są materiały video, do których dostęp uzyskuje się po dodaniu kursu do swoich kursów. Natomiast wciąż nie są to dane wrażliwe, których wyciek stanowi problem. Ochrona przed nieautoryzowanym i złośliwym pobieraniem materiałów video powinna zostać rozbudowywana

przede wszystkim na poziomie walidacji danych, logiki aplikacji oraz autoryzacji, a nie poprzez rozdzielanie usługi przechowywania plików na mikrousługi.

Współdzielenie jednej instancji MinIO przez wiele serwisów upraszcza zarządzanie infrastrukturą oraz redukuje zużycie zasobów bez negatywnego wpływu na bezpieczeństwo systemu. Separacja danych na poziomie bucketów zapewnia logiczne oddzielenie zasobów poszczególnych serwisów, co jest wystarczające w kontekście publicznych zasobów multimedialnych. Ewentualne przejście na architekturę z wieloma instancjami MinIO mogłoby być rozważone w przyszłości jedynie w przypadku znaczącego wzrostu ruchu lub wprowadzenia bardziej restrykcyjnych wymagań bezpieczeństwa dla materiałów edukacyjnych.



Rysunek 21. Architektura Kubernetes

Rozdział 4

Weryfikacja i demonstracja działania systemu

Weryfikacja systemu obejmuje sprawdzenie założeń pracy, ze szczególnym uwzględnieniem aspektu wielonajemcowości oraz separacji danych. Prezentacja zawiera zarówno operacje związane z zarządzaniem platformą, jak i typowe scenariusze użytkowników końcowych.

4.1 Środowisko testowe

Aplikacja została uruchomiona na systemie Linux z jądrem 6.17.8. Wykorzystano Minikube w wersji 1.37.0, gdzie każdy mikroserwis posiada jedną replikę z przydzielonym 768Mi RAM oraz 250m CPU. Dostęp do aplikacji klienckiej odbywa się za pomocą przeglądarki Chrome 142. W systemie istnieje utworzona platforma oznaczona jako kreator kursów, nie znajdują się w niej żadne platformy Najemców. Grafiki zostały uzyskane z serwisu Unsplash na mocy licencji Unsplash Licence[1]. Logami tworzonych platform będą logo Wydziału Elektrycznego PW oraz logo PW.

4.2 Demonstracja kluczowych funkcjonalności

Po wejściu na adres *coursy.com* wyświetla się strona główna kreatora platform. Logo oraz zdjęcie w tle zostało wcześniej dostosowane. Po wejściu w kod źródłowy strony można znaleźć pełną strukturę HTML, co oznacza, że widok został wyrenderowany po stronie serwera oraz dostarczony do przeglądarki.

```
<!-- Fragment navbaru -->
<nav>
  <a href="/">Home</a>
  <a href="/contact">Contact</a>
</nav>

```

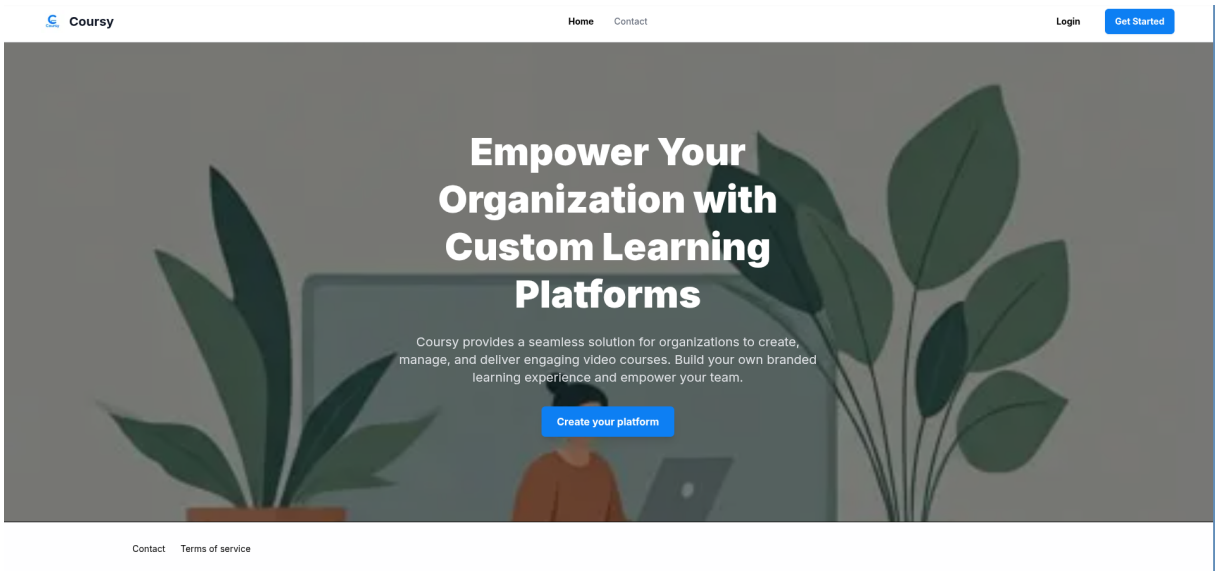
[1] https://unsplash.com

```
<!-- Przyciski logowania i rejestracji -->
<div>
  <div>
    <a href="/login">Login</a>
    <a href="/signup">Get Started</a>
  </div>
</div>

<!-- Fragment śtreci strony łgównej -->
<div>
  <h1>
    Empower Your Organization with Custom Learning Platforms
  </h1>

  <a href="/signup">
    <span>Create your platform</span>
  </a>
</div>
```

Listing 4. Fragment kodu źródłowego strony głównej



Rysunek 22. Strona główna kreatora platform

Strona główna kreatora platform posiada analogiczną strukturę, co strona główna tworzonych platform. Warto w tym miejscu przypomnieć, że kreator platform również jest zapisany jako platforma w systemie, ze specjalnym *platform_id* ustawionym na NULL. Dzięki temu możliwe było wykorzystanie widoków pierwotnie zaprojektowanych dla widoku platform. Dodatkowo można zauważyć, że kreator platform również posiada ustawioną konfigurację. Jest to obserwowalne poprzez skonfigurowaną treść nagłówka, przycisku *CTA*, oraz kolor główny ustawiony na niebieski.

Przy pomocy formularza rejestracyjnego utworzono konto Najemcy „Najemca A” z następującymi danymi:

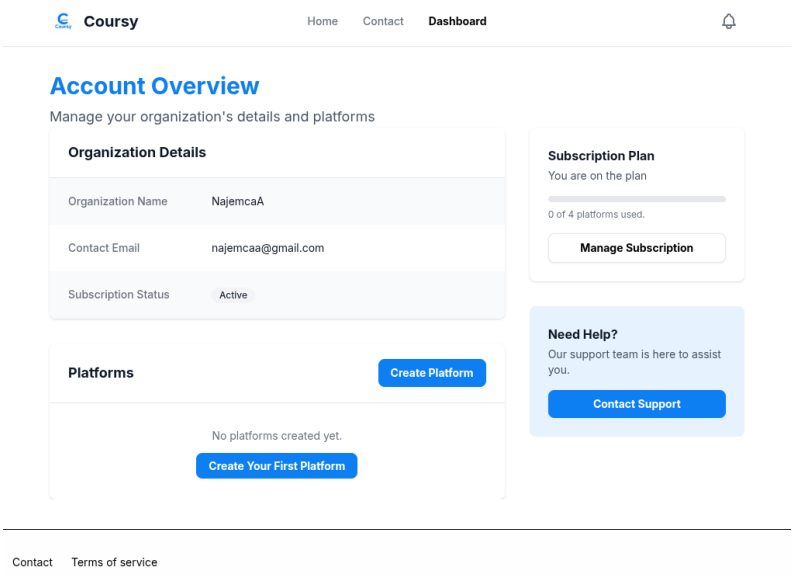
- *First name* - NajemcaA
- *Last name* - NajemcaA
- *Email* - najemcaa@gmail.com
- *Password* - pa$$w0RD

Listing 5 przedstawia żeton JWT, który został przypisany po zalogowaniu w systemie. Można na nim zaobserwować prawidłowy adres email nadawcy oraz wartości wygenerowane przez system. Pole *platformId* posiada przypisaną wartość NULL, co jest poprawną interpretacją zarejestrowanego konta w kreatorze kursów.

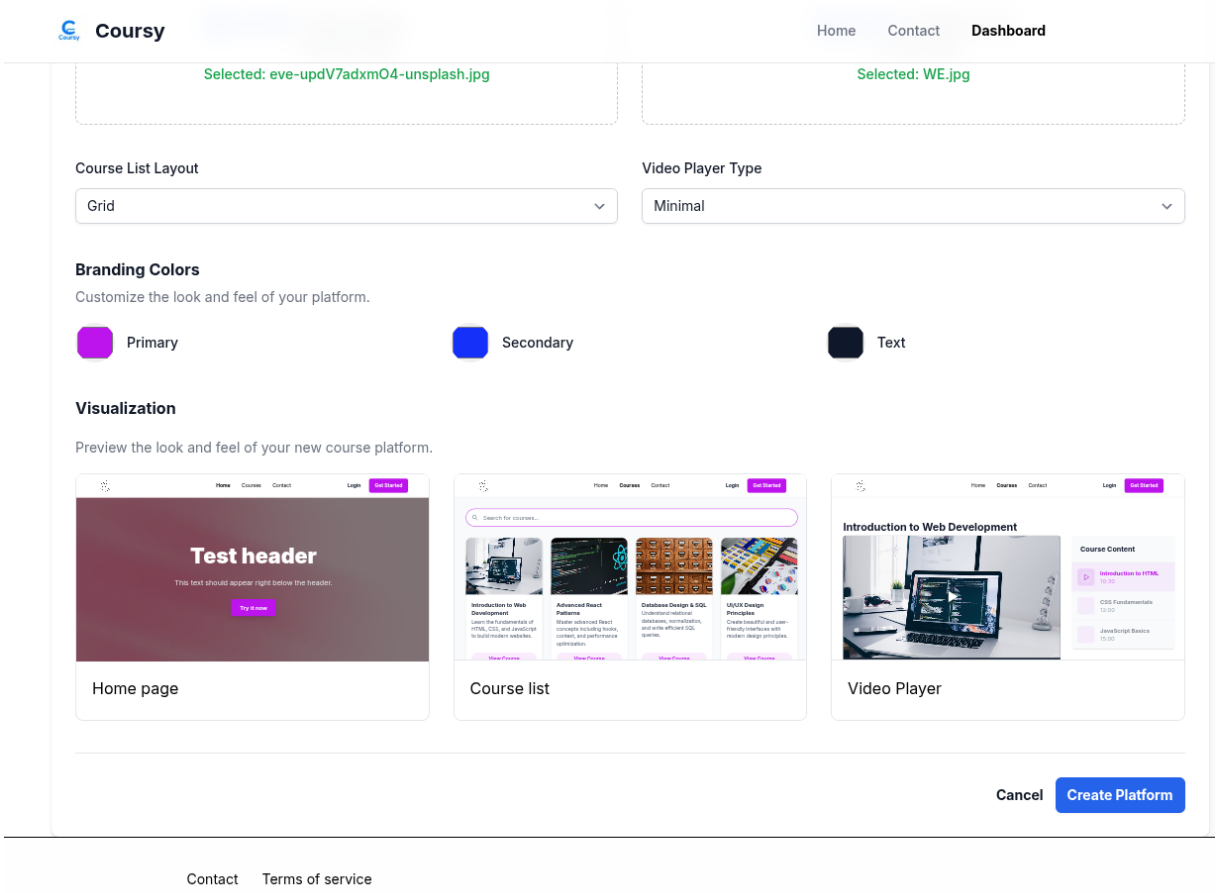
```
{
  "sub": "najemcaa@gmail.com",
  "id": "0b403ad5-662b-4538-a24a-7ce0a8a00aa1",
  "platformId": null,
  "role": "ROLE_TENANT",
  "iat": 1767035476,
  "exp": 1767036376
}
```

Listing 5. Struktura JWT dla utworzonego konta Najemcy

Dashboard przedstawiony na rysunku 23 prawidłowo prezentuje dane Najemcy w głównej części widoku. Dolna zakładka służy do wyświetlania platform utworzonych przez Najemcę. Aktualnie prezentowana jest pusta lista, ponieważ Najemca nie utworzył jeszcze żadnej platformy. W prawej części strony znajdują się dwa widgety, które nie posiadają implementacji i pełnią funkcję czysto demonstracyjną.



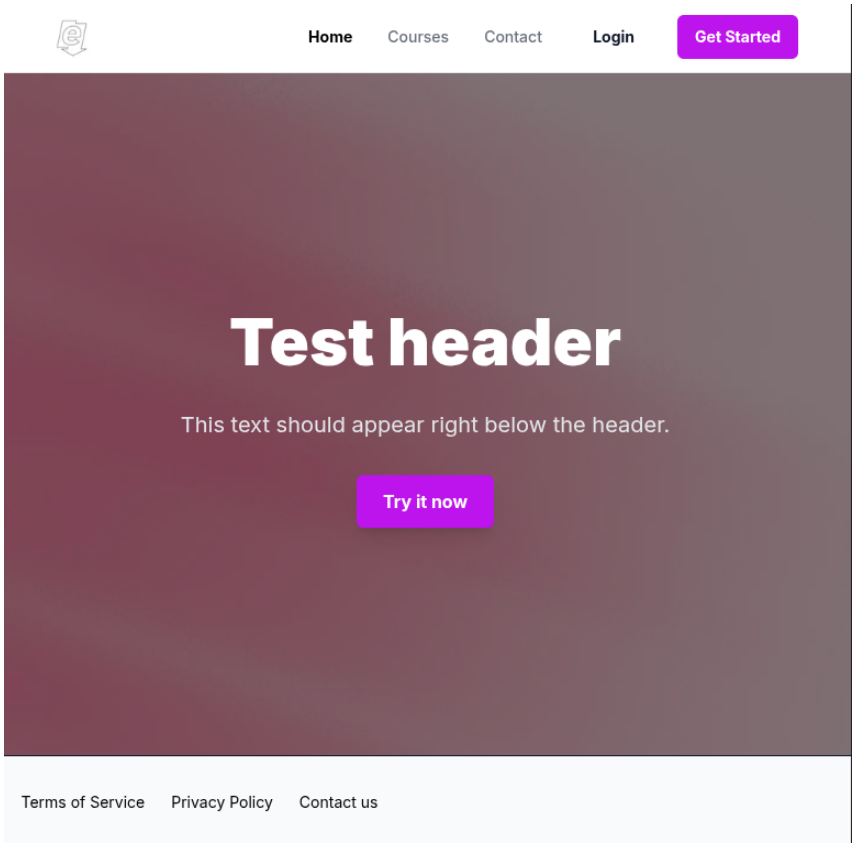
Rysunek 23. Dashboard Najemcy



Rysunek 24. Fragment formularza w kreatorze platform

Rysunek 24 przedstawia fragment formularza tworzenia platform. Na końcu formularza prezentowana jest wizualizacja platformy z aktualną konfiguracją w czasie rzeczywistym. Wizualizacja składa się z widoków oraz komponentów faktycznie używanych na tworzonej platformie. Zostały one przeskalowane przy pomocy właściwości *zoom* w CSS oraz wyłączono w nich wszystkie event listenery. W systemie utworzono platformę o następujących danych:

- *Platform Title* - Test platform A
- *Subdomain* - test-platform-a
- *Description* - This is description of Test Platform A.
- *Hero Title* - Test header
- *Hero Subtitle* - This text should appear right below the header.
- *Call To Action* - Try it now



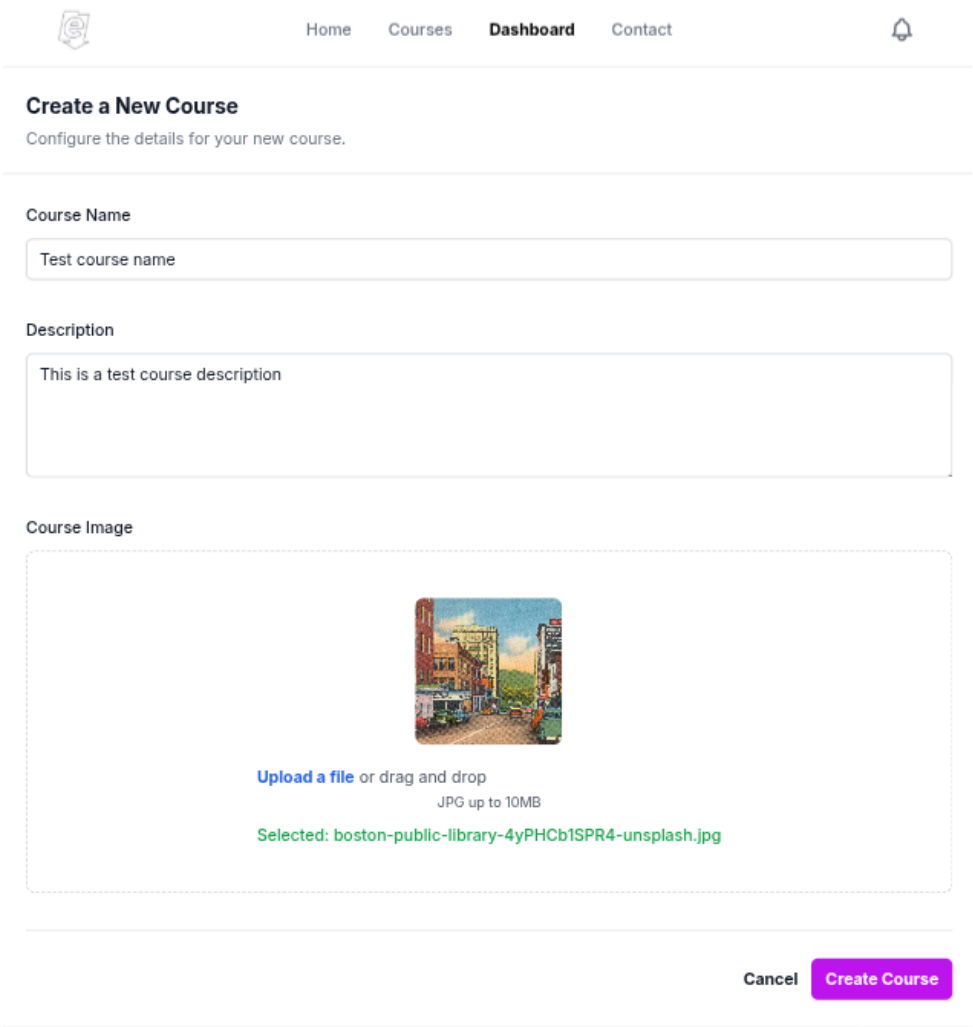
Rysunek 25. Strona główna utworzonej platformy

Po przejściu na adres *test-platform-a.coursy.com* wyświetla się strona główna zgodnie z ustaloną konfiguracją. Prawidłowo wyświetlają się obrazki loga (w lewym górnym rogu) oraz tła. System w poprawny sposób zapisał podane pliki oraz obsługuje wysyłanie ich pod odpowiednimi endpointami. Paleta kolorów platformy jest zgodna z wartościami wybranymi poprzednio na formularzu. Treść strony głównej również jest prawidłowa.

Listing 6 prezentuje strukturę JWT dla zalogowanego właściciela platformy. Należy zwrócić uwagę na nowe *id*, *platformId* oraz *role*. Ten żeton reprezentuje konto właściciela platformy, które zostało utworzone na podstawie konta Najemcy tworzącego tę platformę.

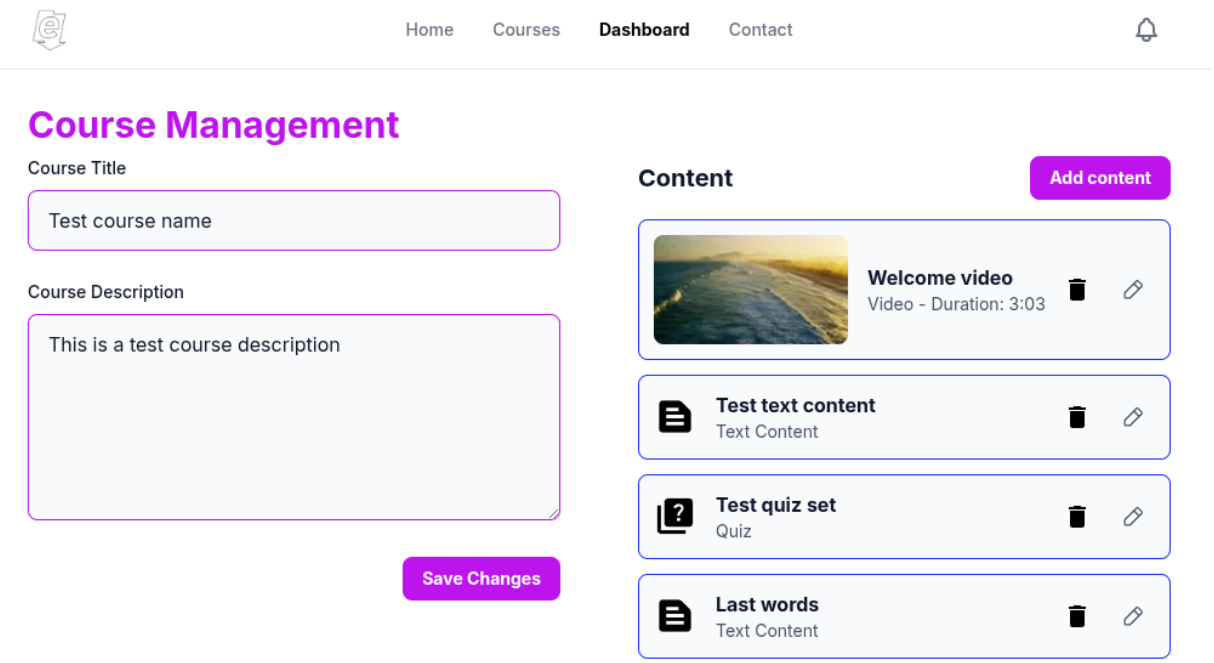
```
{
  "sub": "najemcaa@gmail.com",
  "id": "e3b05212-367e-49d2-8ca6-82f7862bdd3d",
  "platformId": "11c6488f-db75-4bea-a8eb-9ab437f73687",
  "role": "ROLE_PLATFORM_OWNER",
  "iat": 1767039132,
  "exp": 1767040032
}
```

Listing 6. Struktura JWT dla utworzonego konta Najemcy na platformie



Rysunek 26. Tworzenie kursu w platformie

Rysunek 26 przedstawia formularz tworzenia nowego kursu w platformie. Właściciel platformy może uzupełnić dane kursu, czyli jego tytuł, opis, oraz zdjęcie. Wynikiem wysłania formularza jest utworzenie nowego kursu, który nie posiada żadnych materiałów. W celach demonstracyjnych do właśnie utworzonego kursu dodano materiały: film, materiał tekstowy, quiz, oraz drugi materiał tekstowy, pełniący funkcję zakończenia kursu. Wszystkie formularze dodawania materiałów dostępne są z poziomu dashboardu zarządzania kursem.



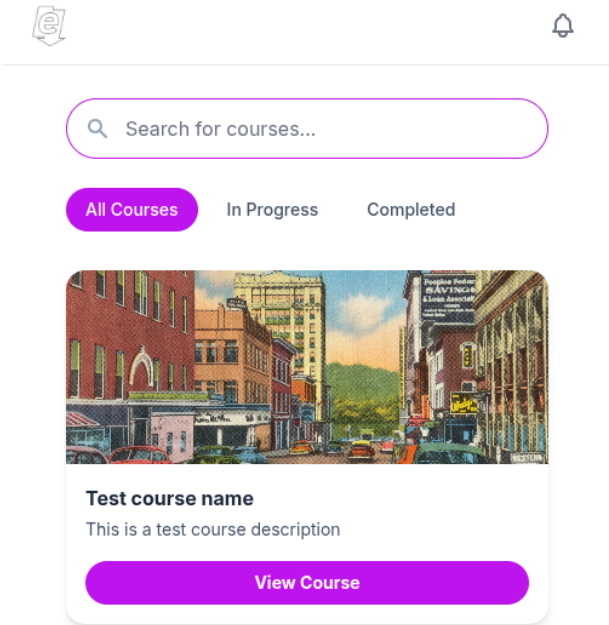
Rysunek 27. Zarządzanie kursem

Na rysunku 27 zaprezentowano dashboard służący do zarządzania kursem. Lewa część widoku zapewnia możliwość zmiany danych kursu. Druga część poświęcona jest zarządzaniu materiałami. W tym miejscu można dodawać materiały edukacyjne, usuwać je, lub wchodzić do zakładek edycji danego materiału. Kolor niebieski w tej części strony jest kolorem dodatkowym zdefiniowanym podczas tworzenia platformy w kreatorze.

Na platformie utworzono konto Użytkownika o adresie email `usera@email.com`. Listing prezentuje strukturę żetonu JWT wygenerowanego przez system dla tego konta. Należy zauważyć, że *platformId* jest identyczny jak w przypadku konta właściciela platformy, a system prawidłowo nadał rolę użytkownika platformy.

```
{
  "sub": "usera@email.com",
  "id": "ca784ef1-6b4f-4f8f-925b-fcbeb708b31f",
  "platformId": "11c6488f-db75-4bea-a8eb-9ab437f73687",
  "role": "ROLE_PLATFORM_USER",
  "iat": 1767073185,
  "exp": 1767074085
}
```

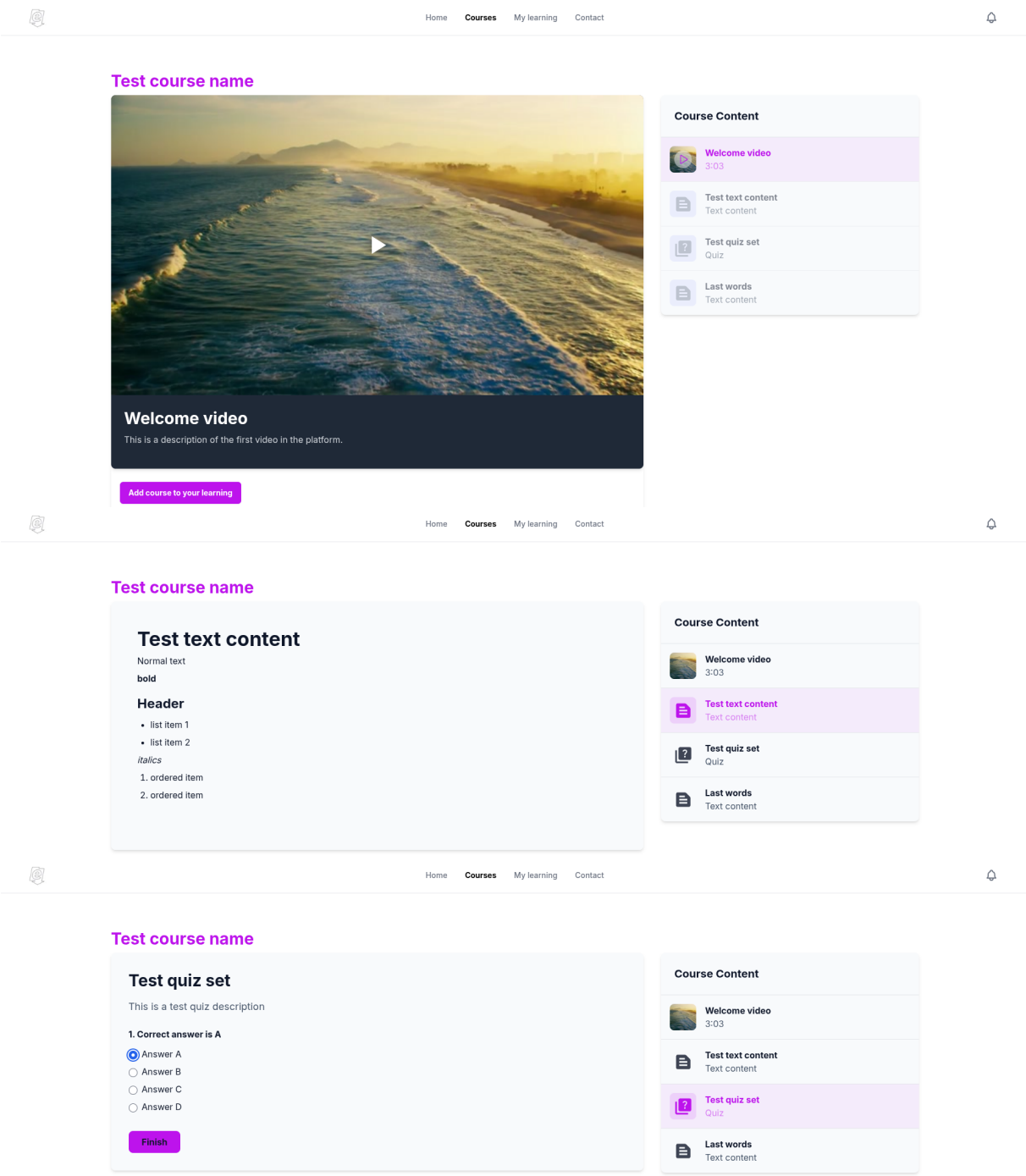
Listing 7. Struktura JWT dla utworzonego konta użytkownika na platformie



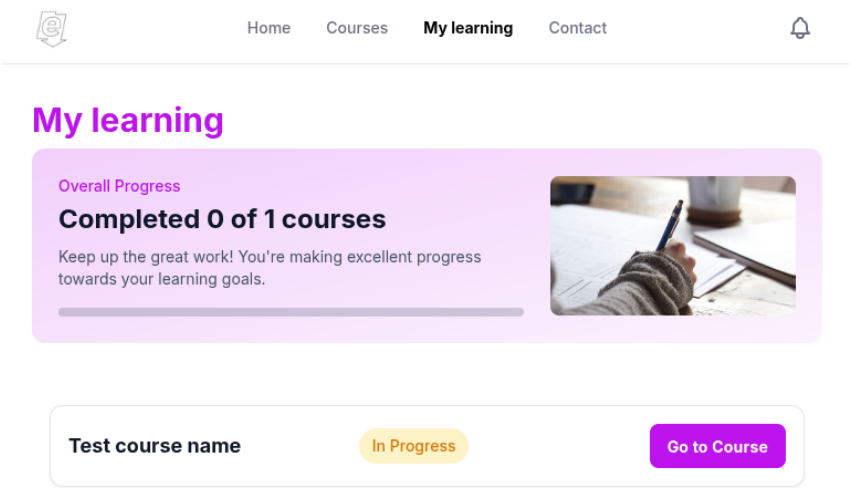
Rysunek 28. Lista kursów

Lista kursów prawidłowo prezentuje kurs, który został dodany przez właściciela. Ten widok jest dostępny dla wszystkich użytkowników platformy, nie tylko dla zalogowanych (czyli gości).

Na rysunku 29 zaprezentowano widok kursu. Materiał video jest wyświetlony w trybie podglądu kursu, ponieważ Użytkownik jeszcze nie dodał go do swojej listy. Aby odblokować pozostałe materiały, należy użyć przycisku „Add course to your learning". Strona w sposób poprawny dostosowuje się do prezentowanego rodzaju materiału oraz aplikuje style zawarte w konfiguracji platformy. Po prawej stronie wyświetlana jest lista wszystkich materiałów w kursie, pełniąc rolę nawigacyjną. Warto zaznaczyć, że na widoku materiału video, który jest wyświetlany w trybie podglądu kursu, prawa część strony posiada wyszarzone elementy. Sygnalizuje to Użytkownikowi, że aby uzyskać do nich dostęp, musi dodać kurs do swojej listy kursów.



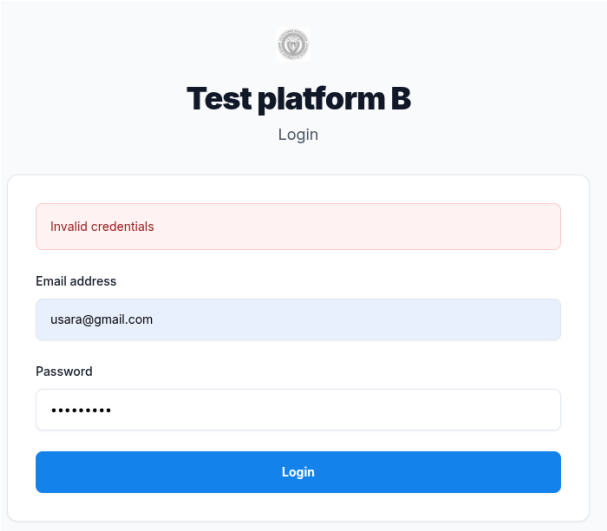
Rysunek 29. Widoki różnych typów materiałów



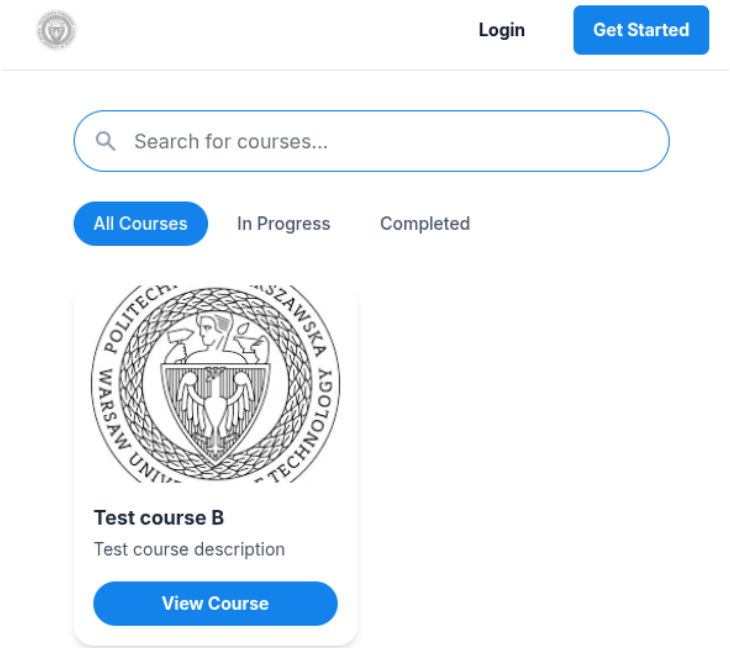
Rysunek 30. Dashboard użytkownika

Na dashboardzie Użytkownika wyświetla się kurs, który został właśnie dodany do listy jego kursów. W liście na dole widoczny jest tytuł kursu, stan (aktualnie jest to „w trakcie"), oraz przycisk służący do nawigacji. Główną część dashboardu zajmuje baner informujący o statusie ukończenia wszystkich kursów.

Powyższy scenariusz związany z utworzeniem platformy, dodaniem kursu oraz obejrzeniem go przez Użytkownika dowodzi, że najważniejsze funkcjonalności platformy zostały spełnione. W celu weryfikacji separacji platform został utworzony drugi Najemca „NajemcaB", druga platforma „Test platform B" oraz kurs „Test course B". Rysunek 31 prezentuje nieudaną próbę logowania na platformę B przez Użytkownika A. System sprawdził, że Użytkownik A posiada przypisane *platformId* inne niż platforma B, więc nie zezwala na zalogowanie się. Dodatkowo rysunek 32 prezentuje kursy na platformie B, które są inne i nie wymieszane z kursami na platformie A.



Rysunek 31. Nieudana próba zalogowania się Użytkownika A na platformie B



Rysunek 32. Kursy dostępne na platformie B

Rozdział 5

Podsumowanie i wnioski

Zbudowany system Coursy to w pełni działająca aplikacja mikroserwisowa z podziałem na frontend oraz backend. Mikroserwisy zostały skutecznie zaimplementowane w sposób w wysokim stopniu niezależny. Wszystkie serwisy posiadają osobną lokalną logikę biznesową, reguły autoryzacyjne oraz przydzieloną bazę danych. Widoki aplikacji zostały utworzone w sposób hybrydowy, wykorzystując renderowanie po stronie serwera tam, gdzie to możliwe. Wszystkie zasoby dostępne w systemie posiadają przypisany *platformId*, który zapewnia wielonajemcowość.

Podział odpowiedzialności biznesowej i wydzielenie mikroserwisów zostały wykonane w większości poprawnie. Mikroserwisy w znacznej części przypadków nie muszą komunikować się ze sobą oraz są samowystarczalne. Jedyną skomplikowaną operacją jest utworzenie nowej platformy, która wymaga współpracy 4 serwisów. Oprócz tego przypadki użycia są realizowane tylko w jednym serwisie. Mikroserwisami, których prawidłowość rozdziału może być kwestionowalna, są *Users* oraz *Auth*.

Przez to, że przechowywanie haseł odbywa się po stronie *Auth*, każde utworzenie konta wymaga komunikacji pomiędzy serwisami. Z drugiej strony role użytkowników są zaimplementowane w *Users*, więc każde wygenerowanie żetonu JWT również wymusza komunikację. Jest to problem architektoniczny, który nie posiada jednoznacznie dobrego rozwiązania. Połączenie tych serwisów w jeden większy serwis spowodowałoby pomieszanie odpowiedzialności. Zarządzanie użytkownikami, w tym prace administracyjne, nie zostałyby oddzielone od operacji tworzenia, odświeżania oraz blokowania żetonów JWT. Serwis stałby się bardziej obszerny, co mogłoby również wpłynąć na prędkość działania. Aktualna implementacja serwisu *Auth* posiada ściśle określone odpowiedzialności, dzięki czemu jest bardziej niezawodny oraz utrzymuje stałą wydajność. Jednocześnie serwis *Users* może ewoluować, wprowadzać nowe funkcjonalności i podlegać refaktoryzacji bez ryzyka wpływu na krytyczne operacje przypisywania żetonów. Należy wziąć pod uwagę, że *Auth* jest najważniejszym z technicznego punktu widzenia serwisem, ponieważ większość przypadków użycia w systemie wymaga autentykacji. Problem komunikacji pomiędzy tymi dwoma serwisami można częściowo rozwiązać poprzez przechowywanie kopii ról użytkowników w warstwie cache w *Auth*. Nie rozwiązuje to jednak problemu całkowicie, ponieważ *Auth* wciąż będzie musiał okresowo lub na żądanie odpytywać *Users* celem synchronizacji. Natomiast całkowite przeniesienie ról do serwisu *Auth* również generuje

problemy. W backendzie został zaimplementowany moduł zarządzania użytkownikami, w który wchodzi między innymi zmiana roli użytkownika przez administratora systemu. Jeżeli role będą przechowywane po stronie *Auth*, serwis *Users* wciąż będzie musiał wykonywać żądania wewnętrzne do tego mikroserwisu.

Biorąc pod uwagę podane przykładowe rozwiązania, można przyjąć, że aktualny rozdział na *Auth* oraz *Users* jest akceptowalnym rozwiązaniem architektonicznym. Komunikację można by zoptymalizować poprzez implementację warstwy cache, która odciążyłaby częściowo *Users* przy komunikacji dotyczącej pobierania roli użytkownika. Oprócz tego do implementacji bazy danych serwisu *Auth* mógł zostać użyty Redis. Tabela *RefreshToken* przechowuje dane tymczasowo, więc możliwe, że użycie w tym celu Redis lub szybszego rozwiązania niż standardowa tabela Postgres mogłoby być korzystne.

Wdrożenie mikroserwisów z wykorzystaniem API Gateway okazało się nie być trudnym procesem. Ponieważ każdy serwis posiada automatycznie przypisany adres w klastrze, nie zachodzi żaden problem związany z komunikacją. Żądania prawidłowo przechodzą przez API Gateway do środka aplikacji, a serwisy mogą komunikować się bezpośrednio pomiędzy sobą. Dodatkowo wprowadzenie orkiestracji Kubernetes ułatwia zarządzanie już uruchomionym systemem.

Wielonajemcowość oparta o przechowywanie danych w jednej tabeli oraz rozróżnianie zasobów w oparciu o wartość pola *platformId* okazała się być odpowiednim podejściem. Podczas implementowania funkcjonalności systemu to rozwiązanie nie było w żadnym przypadku ograniczeniem, a nawet stanowiło uproszczenie. Backend może jednoznacznie określić przynależność każdego obiektu do konkretnej platformy, co przykładowo upraszcza implementację reguł dostępu.

Jedną z największych trudności implementacyjnych systemu było zaprogramowanie warstwy frontend. Podczas implementacji widoków wyszło na jaw, że komponenty generowane po stronie serwera mają inne możliwości w stosunku do komponentów klienckich. Przykładowo popularna biblioteka Redux[1], w oparciu o którą planowane było zarządzanie globalnym stanem, może funkcjonować jedynie w środowisku klienckim. Przykładów takich technologii szeroko używanych w React i niebędących obsługiwanymi po stronie serwera jest wiele, co wymusiło zmianę założeń oraz implementacji pewnych rozwiązań. Co więcej, w niektórych przypadkach ten sam problem musiał zostać zaimplementowany osobno dla obu środowisk. Przykładem może być klient do obsługi wywołań API, automatycznie dołączający JWT do żądań. Po napisaniu większości kodu dla widoków klienckich okazało się, że ten klient oparty o Axios[2] nie jest w stanie obsługiwać ciasteczek po stronie serwerowej. W tym przypadku konieczne było odwołanie do natywnego *fetch*.

Wnioskiem płynącym z tego doświadczenia jest konieczność starannego planowania architektury aplikacji wykorzystującej hybrydowe renderowanie, ze szczególnym uwzględnieniem różnic między środowiskiem serwerowym a klienckim już na etapie projektowania. Kluczowe jest szczegółowe zaplanowanie struktury widoków oraz świadome podjęcie decyzji o miejscu renderowania poszczególnych komponentów. Decyzja ta powinna być podejmowana w oparciu o wymagania funkcjonalne

[1] https://redux.js.org
[2] https://github.com/axios/axios

danego widoku, a nie o dostępność konkretnych rozwiązań technologicznych. W przypadku aplikacji wykorzystujących wyłącznie renderowanie po stronie klienckiej lub serwerowej zestaw dostępnych technologii jest jasno określony i dobrze udokumentowany. Natomiast w podceleejściu hybrydowym, gdzie wykorzystywane są rozwiązania z obu środowisk, konieczna jest dogłębna znajomość ograniczeń i kompatybilności poszczególnych bibliotek oraz narzędzi, aby uniknąć problemów na etapie implementacji.

System spełnia wszystkie wymagania oraz obsługuje przypadki użycia opisane w rozdziale 2. Oprócz tego aplikacja posiada wiele perspektyw rozwoju. Są to między innymi:

- Konfiguracja navbara i footera w platformie,
- Obsługa administratorów, zarządzanie użytkownikami,
- Dodanie serwisu obsługującego płatności,
- Rozszerzenie możliwości konfiguracji platformy,
- Zaimplementowanie bardziej zaawansowanych modeli subskrypcyjnych, odblokowujących nowe opcje konfiguracyjne,
- Funkcjonalności społecznościowe, na przykład możliwość zamieszczania opinii i komentarzy pod kursami,
- Rozwój dashboardów poprzez wprowadzenie modułów analitycznych,
- Funkcjonalności związane z domeną UX, takie jak powiadomienia lub przypomnienia,
- Rozbudowanie wyświetlania materiałów w kursie

Warto zaznaczyć, że kilka z podanych powyżej przykładów jest już częściowo zaimplementowanych po stronie backendu lub aktualna implementacja uwzględnia możliwości rozwoju. Przykładowo schemat bazy danych serwisu *Platforms* został rozbudowany o tabele związane z konfiguracją navbara, footera oraz opcji platformy. Dodatkowo architektura systemu nie posiada ograniczeń w stosunku do dodawania nowych mikroserwisów, jak na przykład nowy serwis obsługujący płatności. Ponieważ aktualnie serwisy są w znacznym stopniu niezależne, dodawanie nowych nie będzie wpływało na działanie już utworzonych. Ta obietnica została spełniona dzięki prawidłowej implementacji architektury mikroserwisowej, zgodnie z założeniami projektu.

Bibliografia

[1] Challa, V., „Comprehensive Analysis of Modern Application Rendering Strategies: Enhancing Web and Mobile User Experiences", *Journal of Engineering and Applied Sciences Technology*, s. 1–6, grud. 2022. DOI: 10.47363/JEAST/2022(4)248.

[2] IBM. „What is Redis Explained?", IBM, term. wiz. 18 grud. 2025. adr.: https://www.ibm.com/think/topics/redis.

[3] Karabey Aksakalli, I., Çelik, T., Can, A. B. i Teki̇nerdoğan, B., „Deployment and communication patterns in microservice architectures: A systematic literature review", *Journal of Systems and Software*, t. 180, s. 111 014, 2021, ISSN: 0164-1212. DOI: https://doi.org/10.1016/j.jss.2021.111014. adr.: https://www.sciencedirect.com/science/article/pii/S0164121221001114.

[4] Kubernetes. „minikube Documentation", Kubernetes, term. wiz. 18 grud. 2025. adr.: https://minikube.sigs.k8s.io/docs/.

[5] MinIO Inc. „MinIO High Performance Object Storage", MinIO, term. wiz. 17 grud. 2025. adr.: https://www.min.io/.

[6] Miyani, M. „Kotlin vs Java – A Comprehensive Comparison", Bugfender, term. wiz. 18 grud. 2025. adr.: https://bugfender.com/blog/kotlin-vs-java/.

[7] Oyeniran, O. C., Adewusi, A. O., Adeleke, A. G., Akwawa, L. A. i Azubuko, C. F., „Microservices architecture in cloud-native applications: Design patterns and scalability", *Computer Science & IT Research Journal*, t. 5, nr. 9, s. 2107–2124, wrz. 2024, ISSN: 2709-0051. DOI: 10.51594/csitrj.v5i9.1554. adr.: https://fepbl.com/index.php/csitrj/article/view/1554.

[8] Pallavi, G. B. i Jayarekha, P., „Multitenancy in SaaS: A Comprehensive Survey", *International Journal of Scientific & Engineering Research*, lip. 2014, ISSN: 2229-5518.

[9] Perforce Software, „2020 Java Technology Report", JRebel by Perforce, spraw. tech., 2020. term. wiz. 18 grud. 2025. adr.: https://www.jrebel.com/blog/2020-java-technology-report.

[10] Peter, U. i Chernyaeva, A. „Helping Decision-Makers Say Yes to Kotlin", JetBrains, term. wiz. 18 grud. 2025. adr.: https://blog.jetbrains.com/kotlin/2025/11/helping-decision-makers-say-yes-to-kotlin/.

[11] Peyrott, S. E., *JWT Handbook*. Auth0, 2016. adr.: https://auth0.com/resources/ebooks/jwt-handbook.

[12] PostgreSQL Global Development Group. „About PostgreSQL", PostgreSQL, term. wiz. 17 grud. 2025. adr.: https://www.postgresql.org/about/.

[13] Rademacher, F., Sorgalla, J. i Sachweh, S., „Challenges of Domain-Driven Microservice Design: A Model-Driven Perspective", *IEEE Software*, t. 35, nr. 3, s. 36–43, 2018. DOI: 10.1109/MS.2018.2141028.

[14] Rumale, A. S. i Chaudhari, D. N., „Cloud computing: Software as a service", w *2017 Second International Conference on Electrical, Computer and Communication Technologies (ICECCT)*, 2017, s. 1–6. DOI: 10.1109/ICECCT.2017.8117817.

[15] Shallal, Q. i Bokhari, M., „CLOUD COMPUTING SERVICE MODELS: A COMPARATIVE STUDY", *IEEE Network*, s. 16–18, mar. 2016.

[16] Snyk. „JVM Ecosystem Report 2020", term. wiz. 18 grud. 2025. adr.: https://snyk.io/blog/jvm-ecosystem-report-2020/.

[17] Śmiałek, M. i Rybiński, K., *Inżynieria wymagań oprogramowania*. Helion, 2024, ISBN: 978-83-289-1268-7.

[18] Vercel Inc. „Next.js by Vercel - The React Framework", Vercel, term. wiz. 19 grud. 2025. adr.: https://nextjs.org/.

[19] Zhang, F., Sun, G., Zheng, B. i Dong, L., „Design and Implementation of Energy Management System Based on Spring Boot Framework", *Information*, t. 12, nr. 11, s. 457, 2021. DOI: 10.3390/info12110457.